

Audit complet

AtelierMBZ

Documentation exhaustive du fonctionnement de l'application

Version 1.0 — 24 avril 2026

Backend Symfony 7.2 + Frontend Nuxt 3

Genere automatiquement depuis docs/AUDIT-V1/AUDIT-COMPLET/

Audit complet AtelierMBZ v4

Date de l'audit : 24 avril 2026 **Périmètre :** intégralité du code source (backend Symfony + frontend Nuxt + infra) **Objectif :** documenter exhaustivement le fonctionnement de l'application, chaque entité, chaque endpoint, chaque règle métier, chaque subtilité.

Comment lire ce document

Ce document est constitué de 6 chapitres compilés dans cet ordre dans le PDF :

#	Chapitre	Sujet
---	----------	-------

00	Préambule	Ce document, méthode
01	Vue d'ensemble	Ce que fait l'appli, métiers, règles non négociables, subtilités, glossaire
02	Backend — modèle	Entités Doctrine, workflows, listeners, voters, migrations
03	Backend — API	Controllers, endpoints, services, commands, crons, messages
04	Frontend Nuxt 3	Pages, composables, stores Pinia, composants, middlewares, layouts, plugins, tests
05	Transverse	Configuration, sécurité, multi-tenant, notifications, Docker, Caddy, tests

Source des informations

Données collectées par lecture directe du code source à la date d'audit. Aucune information ne provient d'inférence ou de documentation externe non vérifiée. Les chemins de fichiers cités sont relatifs à la racine du dépôt.

Mises à jour

Ce document doit être régénéré après tout changement structurel majeur (nouvelle entité, nouveau workflow, nouveau module). La commande de régénération PDF est :

```
docker compose exec php php bin/generate-audit-pdf.php
```

Le PDF de sortie est placé dans `docs/AUDIT-V1/AUDIT-COMPLET-AtelierMBZ.pdf`.

Chapitre 1 — Vue d'ensemble

1.1 Ce que fait AtelierMBZ

AtelierMBZ est une **application de gestion d'atelier moto** destinée à un **réseau franchisé** (pas un SaaS commercial ouvert : pas de plans tarifaires actifs, pas d'inscription libre). Chaque franchisé est une entité juridique distincte (SIRET propre, TVA intra propre, responsable de traitement RGPD indépendant) liée par un contrat de franchise. Le multi-tenant row-level isole les données par établissement sur une base commune.

L'application couvre **deux grandes activités**, branchées sur la même base :

1. **Atelier mécanique moto** — prise de rendez-vous, réception physique, intervention mécanicien, essai routier, restitution, facturation. C'est le cœur de l'usage quotidien.
2. **Véhicules d'occasion (VO)** — rachat à un particulier, remise en état, mise en vente, dépôt-vente, livre de police, déclaration d'achat SIV, facture VO (régime marge ou normal). Module activable par atelier.

Autour de ces deux activités gravitent : clients, véhicules, catalogue moto et prestations, devis, ordres de réparation signés, photos d'intervention, demandes de travaux complémentaires, gardiennage, absences mécaniciens, comptabilité, notifications multi-canaux (email + SMS + Mercure temps réel), audit, paramétrage par atelier, gestion des rôles métier configurables.

Stade actuel

Beta prod soft : en usage réel avec de vrais clients sur un atelier franchisé. Chaque bug impacte de vraies données. Priorité absolue : consolidation du flux principal RDV → réception → mécanicien → restitution.

Modules en réécriture (non couverts par cet audit en profondeur)

- **Stock** — gestion des pièces détachées : en réécriture
- **Facturation** — refonte en cours

Ces modules existent dans le code, sont documentés dans les chapitres techniques, mais ne doivent **pas être modifiés** sans demande explicite.

1.2 Les métiers utilisateurs

Sept profils principaux interagissent avec l'application. Chacun a un écran d'entrée, des contraintes propres et des pièges à éviter.

Le mécanicien (atelier — tablette/PDA)

- 6 à 10 interventions par jour, mains sales, peu de temps clavier
- Support principal : tablette ou smartphone (mobile-first)
- Écran : `/mecanicien`
- Voit ses RDV du jour (filtrés sur `Mecanicien.userId`)
- Démarre/arrête le chrono d'intervention
- Saisit notes techniques dans `OrdreReparation.mechanic_notes`
- Prend des photos via `PhotoController` (types : `en_cours`, `apres_travaux`, `restitution`, `probleme`)
- Crée des demandes de travaux complémentaires
- Remplit et signe l'essai routier puis le rapport d'intervention
- **Ne saisit jamais** : prix, durée estimée, kilométrage de réception
- **Ne touche jamais** `rdv.commentaire` (réservé à la réception)

Le réceptionnaire (comptoir + téléphone)

- Accueil physique, prise de RDV, accueil client, restitution
- Écran principal : `/planning`
- Prend RDV (`/rdv/new`) : recherche client par nom/plaque, crée si nouveau, affecte pont + mécanicien
- **Réceptionne** la moto (transition `reception`) : saisit le km réel, checkup extérieur, photos, signature OR client via Companion
- Valide les demandes de travaux complémentaires (audité via `AuditService`)
- Restitue (signature client sur rapport)
- Gère no-show, reports, gardiennage

Le responsable d'atelier (chef méca)

- Pilote la production, équilibre la charge, gère urgences
- Écrans : `/workshop`, `/admin/absences`
- Valide les OR rectificatifs (`OrdreReparationPolicy`)
- Supervise gardiennage, retours sous garantie
- `garantieTravauxJours` configurable (défaut 30j)

Le responsable magasin / directeur

- Suit CA, marge, rentabilité par activité (`StatistiquesController`)
- Arbitre remises et gestes commerciaux
- Gère les litiges (escalade T+30min SMS via `NotificationEscalation`)

Le gestionnaire VO (`ROLE_VO_MANAGER`)

- Rachat, remise en état, vente, dépôt-vente
- Écrans : `/vo/rachats/` , `/vo/depots/` , `/vo/remises-en-etat/` , `/vo/livre-police` , `/vo/factures`
- OCR carte grise (`useCarteGrise0cr`), génération PV rachat PDF, inscription LP, DA SIV
- Companion VO (`/public/vo-companion`) pour signature électronique vendeur/déposant
- Verdict vendabilité automatique (`V0DocumentService`) avec motifs hiérarchisés (légal, RGPD, workflow, atelier)

Le comptable (`ROLE_COMPTABLE`)

- Consulte factures atelier et VO
- Enregistre encaissements (`Paiement` : espèces, CB, virement, chèque)
- Vérifie cohérence TVA, exporte FEC
- Suppression facture émise **interdite** : annulation par avoir uniquement

Le super-admin (`ROLE_SUPER_ADMIN`)

- Crée/configure ateliers, groupes, utilisateurs
- Paramètre prestations, grilles tarifaires
- Gère rôles métier paramétrables et permissions granulaires
- Suit l'audit log global
- **Bypass total du `TenantFilter`** : toute action est auditée systématiquement via `AuditService::log()`

Le client final (sans compte)

- Reçoit SMS/email de confirmation RDV
- Signe l'OR sur PDA du réceptionniste via Companion (`/public/companion/{token}`)
- Valide/refuse demande de travaux complémentaires via lien SMS (`/public/demande/{token}`)
- Suit son dossier via page publique (`/public/suivi/{token}`)
- Peut prendre RDV lui-même via `/public/booking`
- Pour le VO : signe via `/public/vo-companion/{token}`
- Tokens jamais en query string, toujours en segment de chemin

1.3 Architecture en bref

Backend

PHP 8.3, Symfony 7.2, API Platform 4.1, Doctrine ORM, PostgreSQL 15, Symfony Workflow (state machines), Lexik JWT, DomPDF + Twig, Symfony Messenger.

Frontend

Nuxt 3 (mode SPA, `ssr: false`), Vue 3 Composition API (`<script setup lang="ts">`), Pinia, Nuxt UI v3, composables maison (`useApi`, `useAuth`, `useFormat`, etc.).

Infrastructure

Docker Compose : services `php`, `nuxt`, `worker`, `caddy`, `postgres`, `mercure`, `mailhog`. Caddy comme reverse proxy. `docker-compose.preprod.yml` pour la préprod dédiée.

Multi-tenant row-level

Toutes les entités opérationnelles ont `atelierId`. `TenantFilter` Doctrine applique `WHERE atelier_id = X` automatiquement, activé par `TenantFilterListener` depuis le JWT (`atelier_id` dans le payload). `TenantSetterListener` pose `atelierId` sur les nouvelles entités. `ROLE_SUPER_ADMIN` bypass total.

Modules par atelier

`ConfigAtelier.featureModules` (JSON) active/désactive des modules par atelier. Modules : `dashboard`, `rdv`, `rdv_siege`, `planning`, `workshop`, `suivi`, `clients`, `or`, `motos`, `devis`, `facturation`, `stock`, `mecanicien`, `absences`, `admin`, `tarifs`, `vo` (désactivé par défaut).

1.4 Séquence workflow atelier — l'ordre est impératif

Prise de RDV (planning)

- Réception physique (km relevé, état, carburant, photos, signature OR client)
- Intervention mécanicien (checkup, notes méca, photos, demandes complémentaires)
- Essai routier (obligatoire, km début/fin, points de contrôle)
 - Rapport d'intervention signé mécanicien
 - Restitution (signature client sur rapport)
 - Facturation

Règles de saisie kilométrage — immuables

- **Prise de RDV** : JAMAIS de km
- **Réception** (`/planning`, transition `reception`) : km réel relevé au comptoir → `RendezVous.kilometrage`
- **Rapport d'intervention** (`RapportIntervention.kilometrageRestitution`) : km saisi par le mécanicien à la fin de l'essai routier

OR vs Rapport d'intervention — deux documents distincts

	OrdreReparation	RapportIntervention
Qui le crée	Système à la réception	Mécanicien à la clôture
Qui le signe	Client (avant intervention)	Mécanicien puis client (à la restitution)
Contenu	Prestations à réaliser, devis, accord client	Travaux réalisés, alertes, km restitution, prochaine révision, essai routier, photos
Figé après signature	OUI (<code>OrdreReparationFreezeListener</code>)	OUI
Table	<code>ordres_reparation</code>	<code>rapport_intervention</code>
Template PDF	<code>ordre_reparation.html.twig</code>	<code>rapport_intervention.html.twig</code>

1.5 Subtilités à connaître

Subtilités métier

- **Mécanicien jamais d'estimation** : ni prix ni temps. Ces champs sont saisis par le réceptionnaire.
- **Notes mécanicien** : vont dans `OrdreReparation.mechanic_notes`. Le champ `RendezVous.commentaire` est réservé à la réception et ne doit jamais être écrasé par le mécanicien.
- **Essai routier obligatoire avant terminer** : zéro exception. Bloqué par `RdvTerminationGuardSubscriber`.
- **OR signé = figé** : seuls Resp. Atelier ou Resp. Magasin peuvent faire un OR rectificatif via `OrdreReparationPolicy::rectify()`.
- **3 modes tarification** : FORFAIT / HORAIRE / SUR_DEVIS (cf. `ModeTarificationTest`).
- **Garantie travaux atelier** : 30j par défaut, configurable dans `ConfigAtelier.garantieTravauxJours`.
- **Garantie légale VO** : 12 mois minimum (Art. L.217-3 CC).

Subtilités VO

- **Sans DA SIV enregistrée, le véhicule est invendable** — bloquant vérifié par `VODocumentService::getPurchaseSaleBlockers()`.
- **Livre de Police immuable** : pas de PUT/PATCH/DELETE (Art. 321-7 CP). Lecture seule côté API.
- **Numérotation LP et factures VO** via `VONumberingService` qui utilise des séquences PostgreSQL (jamais MAX+1, sinon race condition).
- **TVA marge vs TVA normale** sur facture VO : régimes exclusifs, jamais mélangés (Art. 297 A vs Art. 256 CGI).

- **Pièce d'identité et justificatif de domicile** : `RETENTION_YEARS = 0` . Refus d'upload côté API ET côté UI. Transcription dans LP puis destruction.
- **Companion VO** : signatures électroniques par token unique (segment de chemin, pas query string), durée d'expiration configurable.
- **Verdict vendabilité** : exposé par `VODocumentService` avec motifs hiérarchisés (légal, RGPD, workflow, atelier).

Subtilités sécurité

- **Tokens Companion** : non-devinables, longs, en segment de chemin (jamais query string). QR code généré localement (`useQrCode`), jamais via service externe.
- **Photos d'intervention** : appartiennent à l'étape mécanicien, pas à la réception. Endpoint `/api/photos` n'est appelé que depuis `/mecanicien` . Photos de réception viennent du Companion.
- **Rôles/permissions** : ne sont jamais stockés dans le `localStorage` . Le contexte est rechargé depuis le serveur à chaque bootstrap (`useAuth().fetchMe()`).
- `/api/docs` : non public en préprod. Seul `/api/health` est public.
- `setStatut()` **direct interdit** : toujours passer par une transition workflow. Sinon les listeners `RdvWorkflowListener` et le guard d'essai routier sont contournés.
- **Bypass** `ROLE_SUPER_ADMIN` : court-circuite le `TenantFilter` . Toute action super-admin doit être auditée via `AuditService::log()` .

Subtilités RGPD

- Durée de conservation codée dans `VODocument::RETENTION_YEARS` :
- Factures : 10 ans
- CERFA cession, carte grise, non-gage, contrat dépôt-vente, PV rachat, mandat immat, récépissé DA, certificat situation admin, copie permis, CT : 5 ans
- **Pièce d'identité : 0 ans** (à détruire après transcription)
- **Justificatif de domicile : 0 ans**
- **Droit à l'effacement** : exclut les données liées à facture ou OR signé. D'où les colonnes `snap_*` figées au moment de l'émission/signature qui permettent l'anonymisation du client/véhicule sans perdre la traçabilité légale.
- **Anonymisation** = nullification des FK + conservation des snapshots. Jamais de suppression de facture, OR, LP.
- **Cron** `app:rgpd-purge` (1er du mois 3:00) purge clients 3 ans inactifs, devis > 6 mois, audit > 3 ans.
- **Cron** `app:purge-identity-documents` (chaque jour 4:00) détruit les pièces d'identité résiduelles.

Subtilités UX

- Pour chaque fonctionnalité, 5 questions :

1. Quelle donnée peut être remplie automatiquement ? (OCR CG, recherche par plaque, historique, grille tarifaire)
 2. Quelle décision peut être proposée par défaut ? (durée estimée, pont affecté, régime TVA)
 3. Quelle action peut être déclenchée en cascade ? (confirmation rachat → LP + PV + DA SIV)
 4. Quelle info peut être visible sans clic ? (marge, jours stock, docs manquants, verdict vendabilité)
 5. Quel rappel anticipé ? (DA SIV J+10, mandat J-7, non-gage expiré, CT expiré)
- **Si une info est en base, elle ne doit jamais être redemandée. Si une action est déductible, elle doit être proposée.**

1.6 Cadre réglementaire

RGPD

- Minimisation, finalité, durée de conservation par type de document
- Sécurité : chiffrement au repos, accès par rôle, audit trail (`AuditLog`)
- Consentement : opt-in séparé pour marketing, pas besoin pour notifications transactionnelles
- DPA obligatoire avec sous-traitants (SMS, email)

Documents obligatoires côté ACHAT VO

Document	Format	Obligation	Conservation
Certificat de cession (Cerfa 15776*02)	Papier/PDF signé	Art. R322-4 Code route	5 ans
Carte grise barrée + signée	Original	Art. R322-4	5 ans (copie)
Certificat situation administrative (< 15j)	PDF ANTS/HistoVec	Art. R322-4	5 ans
Pièce d'identité vendeur	Vérif visuelle	LP	0 jour — détruire
Justificatif de domicile (< 3 mois)	Si demandé	LP	0 jour — détruire
Contrôle technique (moto > 5 ans)	PV ou vignette	Art. R323-22	5 ans
PV de rachat	Signé 2 parties	Preuve transaction	5 ans

Côté SIV : DA (Cerfa 13751) dans les 15 jours (Art. R322-4). Sans DA enregistrée, véhicule invendable.

Documents obligatoires côté VENTE VO

Document	Conservation
Facture VO (PDF)	10 ans
Certificat de cession vente (Cerfa 15776*02)	5 ans
Récépissé DA + carte grise ancienne barrée	remis à l'acheteur
Certificat situation admin < 15j	5 ans

Document	Conservation
CT (> 5 ans, < 6 mois à la vente)	remis à l'acheteur
Notice garantie légale (L.217-3 + 1641 CC)	10 ans
Mandat immatriculation (Cerfa 13757*03) si habilité SIV	5 ans

Mentions facture VO : régime TVA exclusif — soit marge (Art. 297 A CGI), soit normale (Art. 256 CGI).
Jamais les deux.

Dépôt-vente VO (Art. 1915 Code civil)

- LP enregistre 2 lignes : entrée dépôt + vente
- TVA uniquement sur la commission
- Reversement sous 15 jours max
- Mandat durée définie (souvent 90j), prolongeable
- Pas de DA : le déposant reste propriétaire jusqu'à la vente

Réglementation atelier moto

- Devis obligatoire > 150 € (Art. R.112-1 Code consommation)
- OR signé avant intervention
- Travaux complémentaires : accord client explicite avant exécution (workflow `DemandeTravauxSupp`)
- Facturation (Art. 289 CGI) : numérotation continue, mentions obligatoires
- Garantie légale sur pièces et MO (Art. L.217-3) : min 2 ans pièces neuves
- CT moto obligatoire depuis 15/04/2024 (Décret 2023-974) : 5 ans après 1ère immat, puis tous les 3 ans

Archivage comptable

- 10 ans pour pièces comptables (Art. L.123-22 Code commerce)
- 3 ans pour documents fiscaux hors factures
- FEC : format officiel d'export pour contrôle fiscal (Art. L.47 A-I LPF)

1.7 Règles métier non négociables

1. Le mécanicien n'estime jamais (ni prix ni temps)
2. Le mécanicien ne touche jamais `rdv.commentaire`
3. Essai routier obligatoire avant `terminer` — zéro exception
4. OR signé = figé — seuls Resp. Atelier ou Resp. Magasin peuvent faire un rectificatif

5. Le kilométrage n'est jamais saisi à la prise de RDV — uniquement à la réception
6. 3 modes tarification : FORFAIT / HORAIRE / SUR_DEVIS
7. Demandes complémentaires : workflow avec escalade T+5/10/30min
8. Livre de Police immuable : pas de PUT/PATCH/DELETE
9. DA SIV dans les 15 jours : obligatoire, bloquant pour la revente
10. Garantie travaux atelier : 30j par défaut, configurable dans `ConfigAtelier`
11. Garantie légale VO : 12 mois minimum
12. Toute action sensible est auditée via `AuditService::log()`

1.8 Anti-patterns à éviter

- Hardcoder le nom de l'atelier ("PRO MOTO") dans un template → utiliser `atelier.nom`
- `MAX(numero) + 1` pour numéroté LP/factures → race condition. Utiliser séquences PostgreSQL via `VONumberingService`.
- QR Companion via service externe → fuite token. Générer en local via `useQrCode`.
- `VOPurchase.vehicule` OneToOne → doit être ManyToOne
- PDF silencieux en `try/catch` vide → toujours logger
- Saisie d'ID numérique dans un formulaire utilisateur → utiliser recherche/select
- Token Companion en query string → toujours segment de chemin URL
- Uploader une pièce d'identité → refus côté API (`RETENTION_YEARS = 0`) et côté UI
- Stocker rôles/permissions dans le `localStorage` → recharger depuis le serveur
- `setStatut()` direct au lieu d'une transition workflow → contourne les listeners

1.9 Glossaire

Métier atelier

- **OR** — Ordre de Réparation : document signé par le client avant intervention, figé après signature
- **RDV** — Rendez-vous
- **PDA** — Portable Digital Assistant, tablette utilisée en atelier
- **MO** — Main d'Œuvre
- **FRE** — Frais de Remise en État (sur un VO avant revente)
- **CT** — Contrôle Technique
- **Gardiennage** — Facturation du stockage d'un véhicule non récupéré

Métier VO

- **VO** — Véhicule d'Occasion
- **LP** — Livre de Police, registre légal immuable des achats/ventes (Art. 321-7 CP)

- **DA** — Déclaration d'Achat (Cerfa 13751), obligatoire dans les 15 jours pour un pro
- **SIV** — Système d'Immatriculation des Véhicules (base nationale, gérée par l'ANTS)
- **ANTS** — Agence Nationale des Titres Sécurisés
- **CG** — Carte Grise = Certificat d'Immatriculation
- **CPI** — Certificat Provisoire d'Immatriculation (1 mois, France uniquement)
- **CERFA 13751** — Formulaire de Déclaration d'Achat pro
- **CERFA 13757*03** — Mandat pour démarches d'immatriculation
- **CERFA 15776*02** — Certificat de cession d'un véhicule d'occasion
- **VIN** — Vehicle Identification Number (17 caractères ISO 3779, sans I/O/Q)
- **MEC** — Mise En Circulation (date de 1ère immatriculation)
- **HistoVec** — Service public d'historique d'un véhicule

Régalien

- **RGPD** — Règlement Général sur la Protection des Données
- **DPA** — Data Processing Agreement (sous-traitance RGPD)
- **CNIL** — Commission Nationale de l'Informatique et des Libertés
- **CP** — Code Pénal
- **CGI** — Code Général des Impôts
- **LPF** — Livre des Procédures Fiscales
- **CC** — Code Civil

Compta

- **FEC** — Fichier des Écritures Comptables (format officiel pour contrôle fiscal)
- **TVA** — Taxe sur la Valeur Ajoutée
- **HT** — Hors Taxes
- **TTC** — Toutes Taxes Comprises
- **CA** — Chiffre d'Affaires
- **LRAR** — Lettre Recommandée avec Accusé de Réception

Technique

- **CRUD** — Create, Read, Update, Delete
- **DTO** — Data Transfer Object
- **FK** — Foreign Key
- **JWT** — JSON Web Token
- **API** — Application Programming Interface
- **SIRET** — Identifiant unique d'établissement (14 chiffres)
- **DR** — Demande Rectificative (OR rectificatif)
- **SSE** — Server-Sent Events (utilisé par Mercure)

- **CSP** — Content Security Policy
- **HSTS** — HTTP Strict Transport Security

Parfait! J'ai tous les éléments pour générer le rapport exhaustif. Voici le rapport COMPLET du modèle backend:

RAPPORT EXHAUSTIF — MODÈLE

BACKEND SYMFONY

AtelierMBZ Backend v4 — 24 Avril 2026

1. WORKFLOWS SYMFONY

State Machine: rendez_vous

Type: State Machine | Property: statut | Marking Store: method | Audit Trail: enabled

Places (19)

en_attente → reserve → confirme → reception → en_cours → en_pause → termine → restitue → facture → paye → annule | en_attente_pieces | en_attente_reprise | en_gardiennage | restitue_partiel | no_show

Transitions (29)

Transition	From	To	Guard
reserver	en_attente	reserve	—
confirmer	[en_attente, reserve]	confirme	—
reception	confirme	reception	—
start_travail	reception	en_cours	—
mettre_en_pause	en_cours	en_pause	—
reprendre	en_pause	en_cours	—
terminer	[en_cours, en_pause]	termine	Essai routier valide + Rapport intervention signé mécanicien
restituer	termine	restitue	—
facturer	[termine, restitue, restitue_partiel]	facture	—

Transition	From	To	Guard
payer	facture	paye	—
annuler	[en_attente, reserve, confirme, reception, en_attente_pieces, en_gardiennage]	annule	—
declarer_no_show	[confirme, reception]	no_show	—
no_show	[confirme, reception]	no_show	—
reporter	[reception, no_show]	confirme	—
mettre_en_attente_pieces	[en_cours, en_pause]	en_attente_pieces	—
reprendre_apres_pieces	en_attente_pieces	en_cours	—
mettre_en_attente_reprise	en_cours	en_attente_reprise	—
reprendre_demain	en_attente_reprise	en_cours	—
passer_gardiennage	[termine, en_attente_pieces, restitue_partiel]	en_gardiennage	—
mettre_en_gardiennage	[confirme, reception, en_attente_pieces]	en_gardiennage	—
sortir_gardiennage	en_gardiennage	en_cours	—
restituer_partiel	[en_cours, termine]	restitue_partiel	—
retour_garantie	[restitue, paye]	en_cours	—

State Machine: vo_purchase

Type: State Machine | Property: status | Marking Store: method | Audit Trail: enabled

Places (5)

brouillon → en_stock → en_vente → reserve → vendu

Transitions (7)

Transition	From	To
confirmer	brouillon	en_stock
mettre_en_vente	en_stock	en_vente
retirer_de_la_vente	en_vente	en_stock
reserver	en_vente	reserve
liberer	reserve	en_vente

Transition	From	To
vendre	[en_stock, en_vente, reserve]	vendu

State Machine: vo_depot

Type: State Machine | Property: status | Marking Store: method | Audit Trail: enabled

Places (4)

brouillon → actif → vendu / restitue

Transitions (3)

Transition	From	To
activer	brouillon	actif
vendre	actif	vendu
restituer	actif	restitue

2. LISTENERS DOCTRINE/ÉVÉNEMENTS (7)

Listener	Événement	Action	Services injectés
RdvWorkflowListener	workflow.rendez_vous.completed.* (confirmer, terminer, attendre_pieces, mettre_en_attente_pieces, declarer_no_show, no_show)	Dispatch messages d'email de notification (SMS/email). Pour terminer, crée/met à jour RapportIntervention draft.	MessageBusInterface RapportIntervention
TenantFilterListener	kernel.request (priority: -10)	Active/désactive le filtre tenant atelier_id en base de données Doctrine selon le JWT. Bypass complet si ROLE_SUPER_ADMIN.	EntityManagerInterface TokenStorageInterface CurrentAtelier
TenantSetterListener	prePersist (Doctrine lifecycle)	Pose automatiquement atelierId sur les nouvelles entités créées si elle a un setter disponible.	CurrentAtelier

Listener	Événement	Action	Services injectés
OrdreReparationFreezeListener	preUpdate (Doctrine lifecycle)	Gèle les modifications de <code>OrdreReparation</code> quand elle passe en statut signé/exécuté/terminé/rectifié. Contrôle strict des champs modifiables.	—
UserSecurityListener	preUpdate , preRemove (Doctrine User)	Garantit qu'on ne peut pas révoquer le dernier super-admin. Bloque l'escalade de rôle.	UserSecurityContext
ApiDebugExceptionListener	kernel.exception	Capture et loggue les erreurs API sur certaines routes (<code>/api/rendez-vous</code> , <code>/api/ordres-reparation</code>). Inclut contexte requête.	LoggerInterface
SecurityHeadersListener	kernel.response	Ajoute en-têtes HTTP de sécurité : <code>X-Frame-Options: DENY</code> , <code>X-Content-Type-Options: nosniff</code> , <code>HSTS</code> , <code>CSP</code> (Report-Only mode).	—

3. SUBSCRIBERS DOCTRINE (6)

Subscriber	Événement	Action	Services injectés
UserMecanicienSyncSubscriber	postPersist , postUpdate , postFlush	Synchronise User ↔ Mecanicien (1 User ↔ 0..1 Mecanicien). Queue utilisateurs, traite en batch lors du postFlush.	UserMecanicienSyncService
UserRoleMetierSyncSubscriber	prePersist , preUpdate (User)	Synchronise les rôles : transcrit <code>RoleMetier</code> en <code>role</code> hérité, pose <code>atelierId</code> résolu, gère <code>accessStatus</code> (<code>pending_validation</code> → <code>inactive</code>), propage <code>archived</code> → <code>null role</code> .	UserRoleMetierSyncService
DevisDateValiditeSubscriber	prePersist (Devis)	Remplace la durée hardcodée (+30j) du <code>Devis</code> par <code>ConfigAtelier.validiteDevisJours</code> de l'atelier courant.	DevisDateValiditeService

Subscriber	Événement	Action	Se
AbsenceValidationSubscriber	prePersist , preUpdate (Absence)	Valide que date_fin ≥ date_début. Vérifie qu'il n'existe pas d'absence chevauchante pour le même mécanicien. Lance ConflictHttpException si conflit.	Al
RdvTerminationGuardSubscriber	workflow.rendez_vous.guard.terminer	Bloque la transition terminer si : (1) Essai routier absent ou non valide, (2) Rapport intervention non signé par le mécanicien.	Er
UserPasswordHashSubscriber	prePersist , preUpdate (User)	Chiffre les mots de passe via PasswordHasher . Efface credentials. Recompute changeset.	U

4. VOTERS (2)

Voter	Attribut	Sujet	Logique
FactureDeleteVoter	DELETE	Facture	☐ Refuse suppression si statut ∈ [EMISE, PAYEE, PARTIELLEMENT_PAYEE, CORRIGEE]. ☐ Permet si statut brouillon/invalidé. (Conformité Art. L.123-22 Code commerce - conservation 10 ans)
VOFactureDeleteVoter	DELETE	VOFacture	☐ Refuse suppression si statut ∈ [emise, payee, partiellement_payee, corrige, annulee]. ☐ Permet si brouillon. (Conformité Art. L.123-22 + TVA CGI)

5. ENTITÉS DOCTRINE (61)

➤ Entité: Atelier

Table: ateliers

Colonne	Type	Nullable	Default	Unique
id	INT	☐	AUTO	☐
nom	VARCHAR(200)	☐	—	—
slug	VARCHAR(100)	☐	—	☐
adresse	TEXT	☐	NULL	—
cp	VARCHAR(20)	☐	NULL	—

Colonne	Type	Nullable	Default	Unique
ville	VARCHAR(100)	☐	NULL	—
telephone	VARCHAR(20)	☐	NULL	—
email	VARCHAR(200)	☐	NULL	—
siret	VARCHAR(20)	☐	NULL	—
tvaIntracom	VARCHAR(30)	☐	NULL	—
logoUrl	VARCHAR(500)	☐	NULL	—
plan	VARCHAR(50)	☐	'starter'	—
actif	BOOL	☐	true	—
configJson	TEXT	☐	NULL	—
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP	—

Relations: —

Métier: Profil franchise multi-tenant. Un atelier = une entité juridique indépendante (SIRET, TVA, coordonnées). Contient configuration : modules activés, taux horaires, marges pièces, durées de garantie, délais relance gardiennage.

Groupes sérialisation: atelier:read, atelier:write

➤ Entité: User

Table: users

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
username	VARCHAR(100)	☐	—
email	VARCHAR(200)	☐	—
prenom	VARCHAR(120)	☐	NULL
nom	VARCHAR(120)	☐	NULL
hashedPassword	VARCHAR(200)	☐	—
role	VARCHAR(50)	☐	'receptionnaire'
isActive	INT	☐	1
roleMetier	FK → RoleMetier	☐	NULL
authProvider	VARCHAR(30)	☐	'local'
googleSub	VARCHAR(191)	☐	NULL

Colonne	Type	Nullable	Default
accessStatus	VARCHAR(30)	☐	'active'
validatedAt	TIMESTAMP	☐	NULL
validatedBy	INT	☐	NULL
lastLoginAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Relations:

- ManyToOne: RoleMetier (ON DELETE SET NULL)
- OneToOne inverse: Mecanicien (userId)

Métier: Compte utilisateur. Authentification locale ou OAuth Google. Liaison multi-tenant via atelierId ET RoleMetier par atelier (cf UserAtelierRole). Synchronisation automatique ↔ Mecanicien si role = ROLE_MECANICIEN.

Groupes sérialisation: user:read, user:write

➤ **Entité:** RendezVous **(Centrale)**

Table: rendez_vous

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
client	FK → Client	☐	NULL
vehicule	FK → Vehicule	☐	NULL
dateRdv	DATE	☐	—
heureRdv	TIME	☐	—
typeIntervention	VARCHAR(200)	☐	—
commentaire	TEXT	☐	NULL
prixEstime	DECIMAL(10,2)	☐	NULL
prixFinal	DECIMAL(10,2)	☐	NULL
tempsEstime	INT (min)	☐	NULL
tempsFinal	INT (min)	☐	NULL
heureDebutTravail	TIMESTAMP	☐	NULL
heureFinTravail	TIMESTAMP	☐	NULL
tempsEffectifMinutes	INT	☐	NULL

Colonne	Type	Nullable	Default
kilometrage	INT	☐	NULL
etatVehicule	TEXT	☐	NULL
photosEtat	TEXT	☐	NULL
pont	FK → Pont	☐	NULL
mecanicien	FK → Mecanicien	☐	NULL
essaiRoutier	OneToOne → EssaiRoutier	☐	NULL
statut	VARCHAR(50)	☐	'en_attente'
motifAnnulation	VARCHAR(50)	☐	NULL
commentaireAnnulation	TEXT	☐	NULL
tokenPublic	VARCHAR(64)	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Relations:

- ManyToOne: Client , Vehicule , Pont , Mecanicien
- OneToOne: EssaiRoutier (inverse)
- OneToMany: OrdreReparation , PhotoIntervention , DemandeTravauxSupp , PieceUtilisee , RapportIntervention (indirect)

Métier: Centre du domaine atelier. Workflows: rendez_vous (state machine). Cycle: prise RDV → réception (saisie km) → intervention mécanicien → essai routier → restitution → facturation. tokenPublic pour Companion client (signature OR).

Groupes sérialisation: rdv:read , rdv:write , ordre:read

Workflows: rendez_vous state_machine

➤ **Entité:** OrdreReparation

Table: ordres_reparation

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
rendezVous	FK → RendezVous	☐	—
numeroOr	VARCHAR(50)	☐	—
typeOr	VARCHAR(50)	☐	'initial'
kilometrage	INT	☐	NULL
etatVehicule	TEXT	☐	NULL
mechanicNotes	TEXT	☐	NULL

Colonne	Type	Nullable	Default
mechanicNotesUpdatedAt	TIMESTAMP	☐	NULL
mechanicCheckup	TEXT (JSON)	☐	'{'}
mechanicCheckupUpdatedAt	TIMESTAMP	☐	NULL
travaux	TEXT	☐	NULL
demandeTravauxSupp	FK → DemandeTravauxSupp	☐	NULL
signatureClient	TEXT	☐	NULL
statut	VARCHAR(50)	☐	'brouillon'
signedSnapshot	JSON	☐	NULL
signedHash	VARCHAR(64)	☐	NULL
signedAt	TIMESTAMP	☐	NULL
signedIp	VARCHAR(45)	☐	NULL
signedUserAgent	VARCHAR(500)	☐	NULL
rectifiedFrom	FK → OrdreReparation (self)	☐	NULL
motifRectification	VARCHAR(100)	☐	NULL
rectifiedBy	INT	☐	NULL
rectifiedAt	TIMESTAMP	☐	NULL
RGPD Snapshots:			
snapClientNom	VARCHAR(200)	☐	NULL
snapClientPrenom	VARCHAR(200)	☐	NULL
snapVehiculePlaque	VARCHAR(20)	☐	NULL
snapVehiculeMarque	VARCHAR(100)	☐	NULL
snapVehiculeModele	VARCHAR(100)	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Métier: Document contractuel signé par client AVANT intervention. Statuts: brouillon → signe → execute/termine → rectifié (si besoin). Figé après signature (listener `OrdreReparationFreezeListener`). Snapshots RGPD permettent anonymisation du client sans perdre l'historique légal.

Groupe s rialisation: `ordre:read`, `ordre:write`, `rdv:read`

➤ **Entit :** `Client`

Table: `clients`

Colonne	Type	Nullable	Default
---------	------	----------	---------

id	INT	☐	AUTO
atelierId	INT	☐	NULL
nom	VARCHAR(100)	☐	—
prenom	VARCHAR(100)	☐	—
telephone	VARCHAR(20)	☐	—
email	VARCHAR(200)	☐	NULL
adresse	TEXT	☐	NULL
notes	TEXT	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP
RGPD:			
consentDate	TIMESTAMP	☐	NULL
consentSource	VARCHAR(100)	☐	NULL
lastActivityAt	TIMESTAMP	☐	NULL
isAnonymized	BOOL	☐	false

Relations:

- OneToMany: `Vehicule`, `RendezVous`

Métier: Contact client. Tracte consentement SMS/email (RGPD). `isAnonymized` = droit à l'oubli partiel (sauf pièces légales). Recherche full-text par nom/prénom.

Groupes sérialisation: `client:read`, `client:write`, `rdv:read`, `devis:read`, `facture:read`, `ordre:read`

➤ **Entité:** `Vehicule`

Table: `vehicules`

Colonne	Type	Nullable	Default	Unique
id	INT	☐	AUTO	—
atelierId	INT	☐	NULL	—
plaque	VARCHAR(20)	☐	—	(atelierId, vin)
marque	VARCHAR(100)	☐	NULL	—
modele	VARCHAR(100)	☐	NULL	—
annee	INT	☐	NULL	—
cylindree	VARCHAR(50)	☐	NULL	—
typeMoto	VARCHAR(50)	☐	NULL	—
typeVariante	VARCHAR(100)	☐	NULL	—

Colonne	Type	Nullable	Default	Unique
denominationCommerciale	VARCHAR(100)	☐	NULL	—
genreNational	VARCHAR(20)	☐	NULL	—
numeroFormuleCg	VARCHAR(20)	☐	NULL	—
client	FK → Client	☐	NULL	—
categorie	FK → CategorieMoto	☐	NULL	—
modeleRef	FK → ModeleMoto	☐	NULL	—

Relations:

- ManyToOne: Client , CategorieMoto , ModeleMoto
- OneToMany: RendezVous
- Uses trait: V0Trait (VO-specific fields)

Métier: Moto. Lien ↔ client (historique). Utilisé trait pour fields VO optionnels (VIN, PMEC, CT, couleur, etc). Recherche par plaque exacte.

Groupes sérialisation: vehicule:read , vehicule:write , rdv:read , client:read , ordre:read

➤ **Entité:** Devis

Table: devis

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
numeroDevis	VARCHAR(50)	☐	—
client	FK → Client	☐	—
vehicule	FK → Vehicule	☐	NULL
dateCreation	TIMESTAMP	☐	CURRENT_TIMESTAMP
dateValidite	DATE	☐	—
statut	VARCHAR(50)	☐	'brouillon'
kilometrage	INT	☐	NULL
totalMoHt	DECIMAL(10,2)	☐	'0.00'
totalPiecesHt	DECIMAL(10,2)	☐	'0.00'
totalHt	DECIMAL(10,2)	☐	'0.00'
totalTtc	DECIMAL(10,2)	☐	'0.00'
remisePourcentage	DECIMAL(10,2)	☐	'0.00'

Colonne	Type	Nullable	Default
remiseMontant	DECIMAL(10,2)	☐	'0.00'
acompteDemande	DECIMAL(10,2)	☐	'0.00'
notesClient	TEXT	☐	NULL
notesInternes	TEXT	☐	NULL
rendezVousId	INT	☐	NULL

Relations:

- ManyToOne: Client , Vehicule
- OneToMany: LigneDevis

Métier: Devis précontractuel (>150€ = obligatoire par loi). Numérotation unique. Validité contrôlée par ConfigAtelier.validiteDevisJours (défaut 30j, override par DevisDateValiditeSubscriber).

Groupes sérialisation: devis:read , devis:write

➤ Entité: Facture

Table: factures

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
numeroFacture	VARCHAR(50)	☐	—
rendezVous	FK → RendezVous	☐	—
client	FK → Client	☐	—
vehicule	FK → Vehicule	☐	NULL
nature	VARCHAR(20)	☐	'facture'
factureOrigine	FK → Facture (self)	☐	NULL
motifCorrection	TEXT	☐	NULL
totalMoHt	DECIMAL(10,2)	☐	'0.00'
totalPiecesHt	DECIMAL(10,2)	☐	'0.00'
totalHt	DECIMAL(10,2)	☐	'0.00'
tvaMo	DECIMAL(10,2)	☐	'0.00'
tvaPieces	DECIMAL(10,2)	☐	'0.00'
totalTva	DECIMAL(10,2)	☐	'0.00'
totalTtc	DECIMAL(10,2)	☐	'0.00'

Colonne	Type	Nullable	Default
Snapshots RGD:			
snapClientNom	VARCHAR(200)	☐	NULL
snapClientPrenom	VARCHAR(200)	☐	NULL
snapVehiculePlaque	VARCHAR(20)	☐	NULL
snapVehiculeMarque	VARCHAR(100)	☐	NULL
snapVehiculeModele	VARCHAR(100)	☐	NULL
statut	VARCHAR(50)	☐	'emise'
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Statuts: emise, payee, partiellement_payee, corrige

Relations:

- ManyToOne: RendezVous, Client, Vehicule, Facture (factureOrigine)
- OneToMany: LigneFacture, Paiement

Métier: Facture légale (numérotation chrono, inégalité interdite). Immuable après émission → avoir seul rectificatif autorisé. Snapshots = conservation 10 ans, anonymisation possible. FactureDeleteVoter bloque suppression.

Groupes sérialisation: facture:read, facture:write

► Entité: Prestation

Table: prestations

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
code	VARCHAR(50)	☐	—
nom	VARCHAR(200)	☐	—
description	TEXT	☐	NULL
categorie	VARCHAR(100)	☐	'entretien'
sousCategorie	VARCHAR(100)	☐	NULL
prixBaseHt	DECIMAL(10,2)	☐	'0.00'
prixBaseTtc	DECIMAL(10,2)	☐	'0.00'
tempsEstimeMinutes	INT	☐	30
delaiInterventionJours	INT	☐	1

Colonne	Type	Nullable	Default
typeTarif	VARCHAR(50)	☐	'forfait'
tauxHoraireApplique	VARCHAR(50)	☐	'standard'
typeVehicule	VARCHAR(50)	☐	'tous'
cyllindreeMin	INT	☐	NULL
cyllindreeMax	INT	☐	NULL
isActive	INT	☐	1
isForfait	INT	☐	0
isPromo	INT	☐	0
prixPromoTtc	DECIMAL(10,2)	☐	NULL
inclutPieces	INT	☐	0
descriptionPiecesIncluses	TEXT	☐	NULL
coutPiecesInclusesHt	DECIMAL(10,2)	☐	'0.00'
margePiecesPourcent	FLOAT	☐	30.0
garantieJours	INT	☐	NULL
necessiteEssai	BOOL	☐	true
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP
updatedAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Relations:

- OneToMany: GrilleTarifaire (cascade), LigneDevis , LigneFacture

Métier: Catalogue de prestations (MO + pièces incluses optionnelles). Tarification par grille (catégorie moto, cylindrée). typeTarif : forfait/horaire/sur_devis. Garantie configurable.

Groupes sérialisation: prestation:read , prestation:write

➤ **Entité:** Pont

Table: ponts

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
nom	VARCHAR(100)	☐	—
typePont	VARCHAR(50)	☐	'moto'
capaciteKg	INT	☐	500

Colonne	Type	Nullable	Default
isActive	INT	☐	1
ordreAffichage	INT	☐	0
mecanicien	FK → Mecanicien	☐	NULL

Relations:

- ManyToOne: Mecanicien
- OneToMany: RendezVous

Métier: Poste de travail atelier. Affectation optionnelle mécanicien. Utilisé pour planification RDV.

Groupes sérialisation: pont:read , pont:write , rdv:read

➤ **Entité:** Mecanicien

Table: mecaniciens

Colonne	Type	Nullable	Default	Unique
id	INT	☐	AUTO	—
atelierId	INT	☐	NULL	—
nom	VARCHAR(100)	☐	—	—
prenom	VARCHAR(100)	☐	—	—
specialites	TEXT	☐	NULL	—
couleur	VARCHAR(7)	☐	'#3b82f6'	—
isActive	INT	☐	1	—
userId	INT	☐	NULL	(user_id)

Relations:

- OneToMany: RendezVous
- OneToOne→: User (by userId, pas FK Doctrine)

Métier: Profil mécanicien. Lien nu userId (pas FK → pas cascade delete). Synchronisé par UserMecanicienSyncSubscriber lors CRUD User.

Groupes sérialisation: mecanicien:read , mecanicien:write , rdv:read , pont:read , absence:read

➤ **Entité:** EssaiRoutier

Table: essai_routier

Colonne	Type	Nullable	Default
---------	------	----------	---------

id	INT	☐	AUTO
atelierId	INT	☐	NULL
rendezVous	OneToOne → RendezVous	☐	—
kmDebut	INT	☐	NULL
kmFin	INT	☐	NULL
dureeMinutes	INT	☐	NULL
distance	DECIMAL(6,2)	☐	NULL
pointsControle	JSON	☐	10 checkpoints
anomalies	TEXT	☐	NULL
actionsCorrectives	TEXT	☐	NULL
signatureMecanicien	TEXT	☐	NULL
realiseAt	TIMESTAMP	☐	NULL
mecanicienId	INT	☐	NULL
observations	TEXT	☐	NULL
statut	VARCHAR(30)	☐	'en_cours'
validatedAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Statuts: brouillon, valide, anomalie_detectee

Relations:

- OneToOne: RendezVous (inverse)

Métier: Test routier OBLIGATOIRE avant clôture RDV (RdvTerminationGuardSubscriber). 10 points contrôle structurés (freinage, direction, suspension, moteur, boîte, embrayage, éclairage, bruits, tableau bord, comportement). Signature mécanicien = validation.

➤ **Entité:** RapportIntervention

Table: rapport_intervention

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
rendezVous	FK → RendezVous	☐	—
statut	VARCHAR(50)	☐	'brouillon'
travauxRealises	TEXT	☐	NULL

Colonne	Type	Nullable	Default
alertes	JSON	☐	NULL
recommandations	TEXT	☐	NULL
prochaineRevisionKm	INT	☐	NULL
prochaineRevisionDate	DATE	☐	NULL
kilometrageRestitution	INT	☐	NULL
garantie	TEXT	☐	NULL
signatureMecanicien	TEXT	☐	NULL
signeMecanicienAt	TIMESTAMP	☐	NULL
signeMecanicienId	INT	☐	NULL
signatureClient	TEXT	☐	NULL
signeClientAt	TIMESTAMP	☐	NULL
signedSnapshot	JSON	☐	NULL
signedHash	VARCHAR(64)	☐	NULL
signedIp	VARCHAR(45)	☐	NULL
rectifiedFrom	FK → RapportIntervention (self)	☐	NULL
motifRectification	VARCHAR(200)	☐	NULL

Statuts: brouillon, en_validation, signe, rectifie

Relations:

- ManyToOne: RendezVous

Métier: Document résumé des travaux réalisés + signature électronique mécanicien + client (à la restitution). Généré auto à terminer RDV via RapportInterventionService. Bloquant pour fin intervention (RdvTerminationGuardSubscriber).

► Entité: DemandeTravauxSupp

Table: demandes_travaux_supp

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
rendezVous	FK → RendezVous	☐	—
description	TEXT	☐	NULL
prestationsDemandees	TEXT	☐	NULL
urgence	VARCHAR(50)	☐	'normal'

Colonne	Type	Nullable	Default
tempsEstime	INT (min)	☐	NULL
prixEstime	DECIMAL(10,2)	☐	NULL
statut	VARCHAR(50)	☐	'en_attente'
notesReceptionniste	TEXT	☐	NULL
decisionClient	VARCHAR(50)	☐	NULL
decisionClientAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP
approvedAt	TIMESTAMP	☐	NULL
approvedBy	INT	☐	NULL
tokenValidation	VARCHAR(64)	☐	(unique)
prestationsChoisies	JSON	☐	[]
photosJustificatives	TEXT	☐	NULL
decisionIp	VARCHAR(45)	☐	NULL
decisionUserAgent	VARCHAR(500)	☐	NULL
signatureClient	TEXT	☐	NULL
signedAt	TIMESTAMP	☐	NULL
orComplementaire	FK → OrdreReparation	☐	NULL

Statuts: en_attente, en_attente_validation, en_attente_decision_client, accepte, refuse

Métier: Travaux complémentaires détectés par mécanicien. Workflow: demande → validation resp atelier → envoi client (SMS + token) → signature client → création OR complémentaire si acceptation.

Groupes sérialisation: demande:read, demande:write, rdv:read

► Entité: VOPurchase

Table: vo_purchases

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
vehicule	FK → Vehicule	☐	NULL
seller	FK → Client	☐	NULL
expert	FK → User	☐	NULL
status	VARCHAR(20)	☐	'brouillon'

Colonne	Type	Nullable	Default
sivStatus	VARCHAR(20)	☐	'a_preparer'
sivReference	VARCHAR(120)	☐	NULL
sivRecordedAt	TIMESTAMP	☐	NULL
Snapshots Companion:			
companionToken	VARCHAR(64)	☐	NULL
companionExpiresAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Statuts VOPurchase: brouillon, en_stock, en_vente, reserve, vendu

Statuts SIV: a_preparer, en_cours, enregistree, rejetee, expiree

Relations:

- ManyToOne: Vehicule, Client (seller), User (expert)
- OneToMany: VORemiseEnEtat, VOLivrePolice, VOFacture, VODocument
- Trait: VOCompanionTrait (companion signature)

Métier: Workflow d'achat VO. Déclaration d'Achat (DA) SIV obligatoire dans 15 jours (bloquant pour revente). Remise en état (VORemiseEnEtat). LP + Facture VO + Mandat SIV.

Groupes sérialisation: vo:read, vo:write

Workflows: vo_purchase state_machine

➤ **Entité:** VODepotVente

Table: vo_depot_ventes

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
vehicule	FK → Vehicule	☐	NULL
deposant	FK → Client	☐	NULL
gestionnaire	FK → User	☐	NULL
prixVenteSouhaite	DECIMAL(10,2)	☐	—
commissionType	VARCHAR(20)	☐	'pourcentage'
commissionValeur	DECIMAL(10,2)	☐	—
dateDebut	DATE	☐	—
dateFin	DATE	☐	NULL

Colonne	Type	Nullable	Default
dureeJours	INT	☐	90
Snapshots Companion:			
companionToken	VARCHAR(64)	☐	NULL
companionExpiresAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Relations:

- ManyToOne: Vehicule, Client (deposant), User (gestionnaire)
- OneToMany: V0RemiseEnEtat, V0LivrePolice, V0Facture, V0Document
- Trait: V0CompanionTrait

Métier: Dépôt-vente (Art. 1915 Code civil). Mandat durée définie (90j défaut). Commission % ou forfait. Reversement net 15j max. TVA comm seule. Immuable après signature (LP).

Groupes sérialisation: vo:read, vo:write

Workflows: vo_depot state_machine

➤ **Entité:** V0RemiseEnEtat

Table: vo_remises_en_etat

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
voPurchase	FK → V0Purchase	☐	NULL
voDepotVente	FK → V0DepotVente	☐	NULL
campaignIndex	INT	☐	1
titre	VARCHAR(180)	☐	'Remise en etat V0'
status	VARCHAR(40)	☐	'a_chiffrer'
priority	VARCHAR(20)	☐	'normale'
diagnosticNotes	TEXT	☐	NULL
workshopNotes	TEXT	☐	NULL
businessNotes	TEXT	☐	NULL
requestedBy	FK → User	☐	NULL
validatedBy	FK → User	☐	NULL
requestedAt	TIMESTAMP	☐	NULL

Colonne	Type	Nullable	Default
validatedAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Statuts: a_chiffrer, a_valider, validee, pieces_a_commander, en_attente_pieces, planifiee_atelier, en_cours, terminee, cloturee, annulee

Priorités: basse, normale, haute, urgente

Relations:

- ManyToOne: V0Purchase, V0DepotVente (XOR check), User (requestedBy, validatedBy)
- OneToMany: V0RemiseEnEtatLigne (travaux), V0RemiseEnEtatPiece (pièces)

Métier: FRE (Frais de Remise en État). Estimation de coûts + liste travaux/pièces. V0MarginService calcule marge RTE temps réel.

► **Entité:** V0Facture

Table: vo_factures

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
numeroFacture	VARCHAR(50)	☐	—
voPurchase	FK → V0Purchase	☐	NULL
voDepotVente	FK → V0DepotVente	☐	NULL
client	FK → Client	☐	—
vehicule	FK → Vehicule	☐	NULL
regimeTva	VARCHAR(10)	☐	'marge'
prixAchatHt	DECIMAL(10,2)	☐	NULL
mentionTvaMarge	BOOL	☐	true
totalHt	DECIMAL(10,2)	☐	'0.00'
totalTva	DECIMAL(10,2)	☐	'0.00'
totalTtc	DECIMAL(10,2)	☐	'0.00'
statut	VARCHAR(20)	☐	'brouillon'
Snapshots RGPD:			
snapClientNom	VARCHAR(200)	☐	NULL
snapVehiculePlaque	VARCHAR(20)	☐	NULL

Colonne	Type	Nullable	Default
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Statuts: brouillon, emise, payee, partiellement_payee, corrige, annulee

Régimes TVA: marge (Art.297A) | normal (Art.256) — JAMAIS MÉLANGER

Relations:

- ManyToOne: VOPurchase, VODepotVente, Client, Vehicule
- OneToMany: Paiement

Métier: Facture VO. Numérotation auto unique. TVA régime exclusif (marge OU normal). Snapshots immuables. VOFactureDeleteVoter bloque suppression.

Groupe s rialisation: vofacture:read, vofacture:write

➤ **Entit :** VOLivrePolice

Table: vo_livre_police

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
numeroOrdre	INT	☐	—
type	VARCHAR(20)	☐	—
dateAcquisition	DATE	☐	—
dateVente	DATE	☐	NULL
Snapshots l�gaux:			
vendeurNom	VARCHAR(200)	☐	NULL
acheteurNom	VARCHAR(200)	☐	NULL

Relations: —

M tier: Registre l gal IMMUABLE (Art. 321-7 Code P nal). 2 lignes par VO : entr e achat + sortie vente. Pas de PUT/PATCH/DELETE API (lecture seule). Num rotation continue sans trous.

Groupe s rialisation: livrepolice:read (lecture seule)

➤ **Entit :** VODocument

Table: vo_documents

Colonne	Type	Nullable	Default
---------	------	----------	---------

id	INT	☐	AUTO
atelierId	INT	☐	NULL
voPurchase	FK → VOPurchase	☐	NULL
voDepotVente	FK → VODepotVente	☐	NULL
type	VARCHAR(50)	☐	—
filename	VARCHAR(300)	☐	—
contentType	VARCHAR(100)	☐	NULL
fileSize	INT	☐	NULL
uploadedAt	TIMESTAMP	☐	CURRENT_TIMESTAMP
uploadedBy	INT	☐	NULL
retentionExpiresAt	TIMESTAMP	☐	NULL

Types: cerfa_cession_achat, cerfa_cession_vente, carte_grise, non_gage, controle_technique, piece_identite (0j), justificatif_domicile (0j), contrat_depot_vente, da_siv, recepisse_da, mandat_immatriculation, facture_vo, pv_rachat, remise_en_etat, notice_garantie, photo_vehicule, autre

Rétention RGPD: 5 ans (défaut) | 0 ans (pièce identité, justificatif domicile → refus upload + destruction après transcription)

Métier: Stockage documents VO. Droits: upload ROLE_VO_MANAGER seul. Pièce d'identité + justificatif = refus côté API + UI (RGPD, 0j rétention).

Groupes sérialisation: vodoc:read, vodoc:write

➤ **Entité:** ConfigAtelier

Table: config_atelier

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
featureModules	JSON	☐	(18 modules par défaut)
tauxHoraireMoStandard	DECIMAL(10,2)	☐	'65.00'
tauxHoraireMoComplexe	DECIMAL(10,2)	☐	'85.00'
tauxHoraireMoExpert	DECIMAL(10,2)	☐	'95.00'
margePiecesStandard	FLOAT	☐	30.0
margePiecesConsommable	FLOAT	☐	50.0
margePiecesPneumatique	FLOAT	☐	25.0

Colonne	Type	Nullable	Default
forfaitMoMinimum	DECIMAL(10,2)	☐	'25.00'
tvaMoTaux	FLOAT	☐	20.0
tvaPiecesTaux	FLOAT	☐	20.0
validiteDevisJours	INT	☐	30
acomptePourcentage	FLOAT	☐	30.0
delaiRelance1JoursOuvres	INT	☐	15
delaiRelance2JoursOuvres	INT	☐	30
delaiProposeGardiennageJoursOuvres	INT	☐	45
delaiProcedureAbandonJoursOuvres	INT	☐	180
tarifGardiennageJournalier	DECIMAL(10,2)	☐	'5.00'
garantieTravauxJours	INT	☐	30
joursFermetureHebdo	JSON	☐	["sunday"]
datesFermetureExceptionnelles	JSON	☐	[]
dureeDefaultMandatJours	INT	☐	90
regimeTvaVoDefault	VARCHAR(10)	☐	'marge'
updatedAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Relations: —

Métier: Configuration par atelier (singleton par atelierId). Contrôle modules actifs, taux horaires, marges, durées, TVA, gardiennage, mandat VO. DevisDateValiditeSubscriber résout validiteDevisJours.

Groupes sérialisation: config:read , config:write

➤ **Entité:** AuditLog

Table: audit_logs

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
userId	INT	☐	NULL
username	VARCHAR(100)	☐	NULL
action	VARCHAR(100)	☐	—
entityType	VARCHAR(100)	☐	NULL
entityId	INT	☐	NULL

Colonne	Type	Nullable	Default
details	TEXT	☐	NULL
ipAddress	VARCHAR(50)	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Métier: Traçabilité actions sensibles. Immutable (append-only). Filtrages: username, action, entityType, ipAddress, date. Paginé 50 items/page max 200.

Groupes sérialisation: — (ROLE_SUPER_ADMIN seul)

➤ **Entité:** Notification

Table: notifications

Colonne	Type	Nullable	Default
id	INT	☐	AUTO
atelierId	INT	☐	NULL
targetUserId	INT	☐	NULL
targetRole	VARCHAR(100)	☐	NULL
targetRoles	JSON	☐	[]
type	VARCHAR(100)	☐	—
severity	VARCHAR(20)	☐	'info'
title	VARCHAR(255)	☐	—
message	VARCHAR(500)	☐	—
actionUrl	VARCHAR(500)	☐	NULL
relatedEntityType	VARCHAR(100)	☐	NULL
relatedEntityId	INT	☐	NULL
readAt	TIMESTAMP	☐	NULL
readBy	INT	☐	NULL
acknowledgedAt	TIMESTAMP	☐	NULL
acknowledgedBy	INT	☐	NULL
isRead	BOOL	☐	false
priority	VARCHAR(50)	☐	'normal'
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Métier: Notifications in-app. Publiées via Mercure (temps réel). Cible rôle/utilisateur/atelierId. Severity: info/warning/error.

Groupes s rialisation: notif:read , notif:write

► Entit : NotificationTemplate

Table: notification_templates

Colonne	Type	Nullable	Default
id	INT	�	AUTO
atelierId	INT	�	�
code	VARCHAR(100)	�	�
channel	VARCHAR(20)	�	�
libelle	VARCHAR(200)	�	�
sujet	VARCHAR(500)	�	NULL
corps	TEXT	�	�
variables	JSON	�	[]
isActive	BOOL	�	true
createdAt	TIMESTAMP	�	CURRENT_TIMESTAMP
updatedAt	TIMESTAMP	�	CURRENT_TIMESTAMP

Channels: email , sms , push

M tier: Templates notifications configurables. Substitution variables. NotificationDispatcher les utilise. Unique par (atelierId, code, channel).

► Entit : NotificationLog

Table: notification_logs

Colonne	Type	Nullable	Default
id	INT	�	AUTO
atelierId	INT	�	�
channel	VARCHAR(20)	�	�
provider	VARCHAR(50)	�	�
templateCode	VARCHAR(100)	�	NULL
toRecipient	VARCHAR(255)	�	�
subject	VARCHAR(500)	�	NULL
status	VARCHAR(30)	�	�

Colonne	Type	Nullable	Default
providerMessageId	VARCHAR(255)	☐	NULL
errorMessage	TEXT	☐	NULL
sentAt	TIMESTAMP	☐	—
deliveredAt	TIMESTAMP	☐	NULL
readAt	TIMESTAMP	☐	NULL
createdAt	TIMESTAMP	☐	CURRENT_TIMESTAMP

Métier: Historique envois emails/SMS. Statuts: sent, delivered, bounced, failed, opened, clicked. Audit immuable.

► Autres entités importantes (lectures compressées)

Entité	Table	Métier bref
LigneDevis	lignes_devis	Ligne devis (MO/pièce). Calcul montants.
LigneFacture	lignes_facture	Ligne facture. Snapshot immuable post-émission.
PieceDetachee	pieces_detachees	Pièce catalogue. Catégorie, prix achat/vente, marge.
PieceUtilisee	pieces_utilisees	Junction RDV ↔ pièce. Quantité, prix vente.
PhotoIntervention	photos_intervention	Photo RDV (en_cours, apres_travaux, restitution, probleme). SHA256, EXIF, annotation JSON.
GrilleTarifaire	grille_tarifaire	Tarif par prestation × catégorie moto × cylindrée.
Paieement	paiements	Encaissement/remboursement facture atelier ou VO. Type opération, mode (espèces/CB/virement/chèque).
ForfaitMO	forfaits_mo	Forfait temps MO. Promo, pièces incluses optionnelles.
Fournisseur	fournisseurs	Fournisseur pièces. Délai livraison, contact.
CommandeFournisseur	commandes_fournisseur	Commande fournisseur. Suivi statut réception.
CommandePiece	commande_piece	Ancienn table → OBSOLÈTE (remplacée par CommandeFournisseur + LigneCommandeFournisseur).

Entité	Table	Métier bref
Absence	absences	Absence mécanicien (période). Validation absence chevauchante via <code>AbsenceValidationSubscriber</code> .
RoleMetier	roles_metier	Rôle métier configurable par atelier. Template parent. Permissions héritées.
RolePermission	role_permissions	Permissions par rôle legacy (super_admin, admin, user, ...). Voter: <code>RolePermissionVoter</code> .
UserAtelierRole	user_atelier_roles	Junction User ↔ Atelier ↔ RoleMetier. Composite PK (userId, atelierId).
RevokedToken	revoked_tokens	JWT révoqués. Table immuable. Vérifié à chaque requête.
AnnulationRdv	annulation_rdv	Historique annulations RDV. Motifs: client_desiste, no_show, piece_non_disponible, etc.
VORemiseEnEtatLigne	vo_remise_en_etat_lignes	Ligne travaux FRE. Prestation liée optionnelle.
VORemiseEnEtatPiece	vo_remise_en_etat_pieces	Pièce FRE. Reference, quantité, statut (en_stock, a_commander, commandee, recue, montee, annulee).
ModeleMoto	modele_motos	Marque + modèle moto. Relations specs techniques.
CategorieMoto	categorie_motos	Catégorie moto (grosse trail, naked, scooter, etc). Utilisée grille tarifs + CT.
MotoTechnicalSpec	moto_technical_specs	Specs techniques moto par génération. JSON: general, moteur, pneumatique, freinage, suspension, systèmes électriques, entretien.
VOCounter	vo_counters	Séquence numérotation VO (LP, factures VO, DA). Compteur par année + atelier.
NotificationProviderConfig	notification_provider_configs	Config providers SMS/email par atelier. Chiffré. Primary/Fallback. Testable.
NotificationEscalation	— (logique)	Escalade auto si non-réponse. Délais: 5/10/30min (configurable).
Module	modules	Catalogue modules (dashboard, rdv, planning, workshop, mécanicien, VO, stock, facturation, absences, tarifs, admin, suivi, clients, motos, devis, ordres, notifications, clauses-legales).

Entité	Table	Métier bref
InterventionType	intervention_types	OBSOLÈTE — Remplacé par Prestation + GrilleTarifaire.
ClauseLegale	clause_legale	Clauses légales (mentions facturation, garanties, conditions générales). Versionning. Effective_from.
RolePermissionEntry	role_permission_entries	Entrée permissions granulaires (PERM_rdv.edit, PERM_facture.delete, etc). Héritage template.

6. MIGRATIONS DOCTRINE (40 versions)

Version	Date	Description
20260101000000	2026-01-01	Bootstrap initial (réservé)
20260416115054	2026-04-16	RGPD: snapshots client/véhicule devis/factures, champs consentement client
20260416171207	2026-04-16	(non analysée)
20260417033458	2026-04-17	(non analysée)
20260417041023	2026-04-17	Notification infrastructure: tables templates, logs, provider configs, notifications
20260417043703	2026-04-17	PhotoIntervention: +type, +sha256, +exif, +takenAt (LOT 2)
20260417043854	2026-04-17	AnnulationRdv: table création/suppression (test)
20260417044250	2026-04-17	(non analysée)
20260417044805	2026-04-17	Gardiennage: +delaiRelance1/2, +delaiProposeGardiennage, +delaiProcedureAbandon, +tarifGardiennageJournalier, +garantieTravauxJours, +joursFermetureHebdo, +datesFermetureExceptionnelles sur ConfigAtelier. RDV: +emplacement_stockage, +photo_stockage_filename, +mis_en_stockage_at/par, +gardiennage_debut_at/par/motif. CommandePiece (test).
20260417045135	2026-04-17	ClauseLegale: table test create/drop
20260417082000	2026-04-17	User: +prenom, +nom (test add/drop)

Version	Date	Description
20260417100000	2026-04-17	RoleMetier + Module + RolePermissionEntry + User.roleMetier FK
20260417110000	2026-04-17	(non analysée)
20260417120000	2026-04-17	(non analysée)
20260417130000	2026-04-17	(non analysée)
20260417150000	2026-04-17	(non analysée)
20260417183000	2026-04-17	(non analysée)
20260418091500	2026-04-18	VOPurchase.vehicule/seller: nullable (test)
20260420170500	2026-04-20	Grille tarifs anciennes → test cleanup (calculs_tarifs, grille_tarifs, temps_interventions)
20260420194500	2026-04-20	(non analysée)
20260421080200	2026-04-21	Paiement: +atelierId, +typeOperation ('encaissement')
20260422081500	2026-04-22	(non analysée)
20260422115453	2026-04-22	(non analysée)
20260422130000	2026-04-22	(non analysée)
20260423090000	2026-04-23	CerfaFieldConfig: table test create/drop
20260423130422	2026-04-23	RapportTechnicien: table (DEBT: probablement ancien/doublon)
20260423130654	2026-04-23	Mecanicien.userId: FK → User (ON DELETE SET NULL)
20260424153000	2026-04-24	(non analysée — current date)

Version	Date	Description
20260604120000	2026-06-04	VO Module: Vehicule (+mileage, +vin, +is_a2_compatible, +date_premiere_mise_en_circulation, +couleur, +registration_cost, +options_and_accessories, +controle_technique_date, +controle_technique_resultat). Puis création VO tables (vo_purchases, vo_depot_ventes, vo_livre_police, vo_documents, vo_factures) + Paiement.vo_facture_id. Droits: ROLE_VO_MANAGER.
20260604130000	2026-06-04	(non analysée)
20260604140000	2026-06-04	(non analysée)
20260604170000	2026-06-04	VOCOUNTER: table création/suppression test
20260604180000	2026-06-04	VORemiseEnEtat + VORemiseEnEtatLigne + VORemiseEnEtatPiece: tables, FK, CHECK (source XOR), cascade delete
20260604190000	2026-06-04	(non analysée)
20260604193000	2026-06-04	(non analysée)
20260604200000	2026-06-04	VOPurchase: +siv_status, +siv_reference, +siv_recorded_at, +siv_notes (DA SIV tracking)
20260604213000	2026-06-04	(non analysée)
20260605093000	2026-06-05	(non analysée)
20260605100000	2026-06-05	Vehicule: +type_variante, +denomination_commerciale, +genre_national, +numero_formule_cg (CARTE GRISE fields)

7. RÉSUMÉ DOMAINE

Cycle métier atelier (workflow RendezVous)

Prise RDV (planning)

- Réception (km, état, photos, signature OR client Companion)
- Intervention mécanicien (notes, checkup JSON, demandes complémentaires)
- Essai routier obligatoire (10 points contrôle, signature mécanicien)
- Rapport intervention (travaux réalisés, km restitution, signature mécanicien + client)
- Restitution (signature client au comptoir)
 - Facturation (numérotation, snapshots RGPD)
 - Paiement (encaissement multi-mode)

Cycle métier VO (workflows VOPurchase + VODepot)

Rachat (VOPurchase):

brouillon → en_stock (DA SIV à_preparer) → en_vente → reserve → vendu
↳ Remise en État (FRE) parallèle
↳ Livre de Police (2 lignes: entrée + sortie)
↳ Facture VO (TVA marge ou normal)
↳ Mandat SIV + DA SIV (15 jours obligatoire)

Dépôt-vente (VODepot):

brouillon → actif → vendu / restitue
↳ Commission % ou forfait
↳ Reversement net 15j max
↳ Durée mandat (90j défaut, renouvelable)

Entités centrales (61 totales)

- **Atelier** (multi-tenant root)
- **User** ↔ Mécanicien (1:0..1 sync auto)
- **RendezVous** (state machine, central hub)
- **OrdreReparation** (figé post-signature, snapshots RGPD)
- **EssaiRoutier** (obligatoire avant clôture)
- **RapportIntervention** (signature mécanicien + client)
- **DemandeTravauxSupp** (escalade client, token validation)
- **Facture** (immuable, voter delete bloqué, snapshots RGPD)
- **VOPurchase** / **VODepotVente** / **VORemiseEnEtat** (VO workflow)
- **VOLivrePolice** (immutable register)
- **VOFacture** (TVA régime exclusif)
- **Notification** / **NotificationTemplate** / **NotificationLog** (stack notifications)
- **ConfigAtelier** (singleton par atelier)
- **Paielement**, **Prestation**, **GrilleTarifaire**, **Devis**, **PhotoIntervention**, etc.

FIN DU RAPPORT

Parfait ! J'ai maintenant suffisamment de contexte pour générer le rapport exhaustif. Laissez-moi compiler tout cela :

RAPPORT EXHAUSTIF — BACKEND

SYMFONY ATELIERMBZ

☐ RÉSUMÉ EXÉCUTIF

Projet : AtelierMBZ v4 (Gestion d'atelier moto franchisé) **Framework** : Symfony 7.2 + API Platform 4.1 **BD** : PostgreSQL 15, Doctrine ORM **Async** : Symfony Messenger **Notifications** : Email + SMS + Mercure (temps réel)

☐ CONTROLLERS (43 au total)

HealthController

But : Endpoint de santé publique (health check).

Méthode	Chemin	Permission	But	Codes
GET	/api/health	Public	Retour {status: "ok", time: ISO8601}	200

AuthController

But : Authentification, gestion des tokens JWT, login/logout, refresh, switch atelier, intégration Google OAuth.

Méthode	Chemin	Permission	But	Codes
GET	/api/auth/google/dev-simulate	Public (DEV)	Simule une authentification Google en mode dev	200, 400
GET	/api/auth/google/url	Public	Retourne l'URL d'authentification Google	200
POST	/api/auth/google/exchange	Public	Échange le code OAuth contre un JWT	200, 401
POST	/api/auth/login	Public	Login classique (username/password)	200, 401
GET	/api/auth/me	ROLE_USER	Retourne l'utilisateur courant + permissions	200
GET	/api/auth/rdv-ateliers	ROLE_USER	Liste des ateliers accessibles pour booking	200

Méthode	Chemin	Permission	But	Codes
POST	/api/auth/switch-atelier	ROLE_USER	Change l'atelier courant (génère nouveau JWT)	200, 400
POST	/api/auth/refresh	Public	Refresh le JWT depuis le refresh token	200, 401
POST	/api/auth/logout	ROLE_USER	Révoque le JWT courant	200

Services injectés : EntityManager, JWTTokenManager, UserPasswordHasher, HttpClient, Mailer, UserRoleMapper, BookingAtelierAccessService

RendezVousController

But : Gestion complète du workflow des RDV (CRUD, transitions, validations métier).

Méthode	Chemin	Permission	But	Codes
POST	/api/rendez-vous	ROLE_USER	Crée un RDV + client/véhicule auto-crées	201, 400
POST	/api/rendez-vous/{id}/transition/{transition}	ROLE_USER	Applique une transition workflow (confirmer, reception, terminer, etc.)	200, 409, 400

Validations métier :

- Essai routier obligatoire avant terminer
- Demandes de travaux complémentaires en attente bloquent la clôture
- Notes mécanicien requises si anomalies à l'essai

OrdreReparationController

But : Gestion des ordres de réparation (OR) signés, rectification, vérification intégrité.

Méthode	Chemin	Permission	But	Codes
POST	/api/or/{id}/rectifier	ROLE_ADMIN	Crée un nouvel OR rectificatif (copie l'original signée)	201, 403
GET	/api/or/{id}/verify-integrity	ROLE_USER	Vérifie que l'OR n'a pas été altéré après signature	200

MecanicienController

But : Espace de travail du mécanicien (tableau, rapports, essais, notes).

Méthode	Chemin	Permission	But	Codes
GET	/api/mecanicien/me	ROLE_USER	Infos du mécanicien courant + statistiques	200
GET	/api/mecanicien/me/rdvs	ROLE_USER	Liste RDV du jour du mécanicien (filtrés userId)	200
PATCH	/api/mecanicien/me/rapport/{orId}	ROLE_USER	Sauvegarde brouillon du rapport d'intervention	200, 400
POST	/api/mecanicien/me/essai-routier	ROLE_USER	Crée un essai routier (km début/fin, points contrôle, anomalies)	201, 400

RapportInterventionController

But : Gestion des rapports d'intervention (création, validation, signature mécanicien/client, signatures, PDF).

Méthode	Chemin	Permission	But	Codes
GET	/api/rdv/{rdvId}/rapport	ROLE_USER	Récupère ou crée le brouillon du rapport	200
PUT	/api/rapport/{id}	ROLE_USER	Update rapport (avant signature)	200, 400
POST	/api/rapport/{id}/essai	ROLE_USER	Crée/update essai routier attaché au rapport	201, 200
POST	/api/rapport/{id}/sign-mecanicien	ROLE_USER	Signe électroniquement par le mécanicien	200, 400
POST	/api/rapport/{id}/sign-client	Public (token)	Signe électroniquement par le client	200, 403
GET	/api/rapport/{id}/pdf	ROLE_USER	Génère et retourne le PDF	200
POST	/api/rapport/{id}/send-email	ROLE_USER	Envoie le rapport par email au client	200
POST	/api/rapport/{id}/rectifier	ROLE_ADMIN	Crée un rapport rectificatif (immuabilité post-sig)	201

PhotoController

But : Upload, stockage, listage des photos d'intervention.

Méthode	Chemin	Permission	But	Codes
POST	/api/photos/upload	ROLE_USER	Upload d'une photo (type: en_cours, apres_travaux, restitution, probleme)	201, 413, 400

Méthode	Chemin	Permission	But	Codes
GET	/api/photos/file/{filename}	ROLE_USER	Sert une photo stockée	200, 404
GET	/api/photos/rdv/{rdvId}	ROLE_USER	Liste photos d'un RDV	200

PublicPhotoController

But : Accès public aux photos via token Companion (sans auth JWT).

Méthode	Chemin	Permission	But	Codes
GET	/api/public/photos/{token}/{filename}	Public (token)	Sert une photo publique attachée au RDV	200, 404

RdvPrestationCatalogController

But : Retourne le catalogue des prestations applicables à un véhicule.

Méthode	Chemin	Permission	But	Codes
GET	/api/rdv/prestations-catalogue	ROLE_USER	Liste prestations filtrées par catégorie véhicule et tarifs	200

SlotController

But : Calcule les créneaux disponibles (ponts × horaires × mécaniciens).

Méthode	Chemin	Permission	But	Codes
GET	/api/slots	ROLE_USER	Creneaux dispo (query: date, duree_estimee, atelierId)	200
GET	/api/creneaux/disponibles	ROLE_USER	Alias legacy pour compatibilité frontend	200

DevisController

But : Gestion des devis (création, envoi, acceptation, refus, conversion en OR, PDF).

Méthode	Chemin	Permission	But	Codes
POST	/api/devis/{id}/envoyer	ROLE_USER	Envoie le devis au client (génère token de signature)	200
POST	/api/devis/{id}/email	ROLE_USER	Envoie le PDF par email	200

Méthode	Chemin	Permission	But	Codes
POST	/api/devis/{id}/accepter	ROLE_USER	Client accepte → crée OR	201, 400
POST	/api/devis/{id}/refuser	ROLE_USER	Client refuse → statut refusé	200
POST	/api/devis/{id}/convertir	ROLE_USER	Force conversion en OR (internal)	201
GET	/api/devis/{id}/pdf	ROLE_USER	Télécharge le PDF	200

PontStatusController

But : État en temps réel des ponts (occupé, libre, maintenance).

Méthode	Chemin	Permission	But	Codes
GET	/api/ponts/status	ROLE_USER	JSON statuts ponts + équipements	200

GardiennageController

But : Gestion du gardiennage (RDV non récupérés, relances, tarifs, commandes pièces).

Méthode	Chemin	Permission	But	Codes
POST	/api/rdv/{id}/declencher-gardiennage	ROLE_USER	Déclenche le gardiennage (crée log, notifications)	200, 400
GET	/api/rdv/{id}/gardiennage-montant	ROLE_USER	Calcule le montant gardiennage (jours × tarif config)	200
GET	/api/rdv/{id}/commandes-pieces	ROLE_USER	Liste pièces commandées en gardiennage	200
POST	/api/rdv/{id}/commandes-pieces	ROLE_USER	Crée une commande de pièces (gardiennage)	201
PUT	/api/commandes-pieces/{id}	ROLE_USER	Update commande	200
POST	/api/commandes-pieces/{id}/recue	ROLE_USER	Marque la pièce reçue	200

MotosLookupController

But : Recherche et autocomplete motos (marque, modèle, specs techniques).

Méthode	Chemin	Permission	But	Codes
GET	/api/motos/marques	ROLE_USER	Liste des marques de moto	200
GET	/api/motos/autocomplete	ROLE_USER	Autocomplete (query: "marque modele")	200

VehiculeLookupController

But : Recherche véhicules par plaque ou VIN.

Méthode	Chemin	Permission	But	Codes
GET	/api/vehicule/{query}	ROLE_USER	Cherche véhicule par plaque/VIN	200

ClientController

But : Gestion des clients (liste, anonymisation RGPD, export données).

Méthode	Chemin	Permission	But	Codes
GET	/api/clients	ROLE_USER	Liste clients de l'atelier (paginé)	200
POST	/api/clients/{id}/anonymize	ROLE_ADMIN	RGPD anonymisation client	200
GET	/api/clients/{id}/export	ROLE_ADMIN	Export JSON données client (RGPD droit à portabilité)	200

ClientStatsController

But : Statistiques clients (count, activité).

Méthode	Chemin	Permission	But	Codes
GET	/api/clients/stats	ROLE_USER	Compte clients actifs, no-show, taux satisfaction	200

StatistiquesController

But : Dashboard KPIs (CA, marge, mécaniciens, rentabilité).

Méthode	Chemin	Permission	But	Codes
GET	/api/statistiques/dashboard	ROLE_USER	KPIs dashboard (CA 30/90j, anomalies, alerts)	200
GET	/api/statistiques/ca	ROLE_USER	Chiffre affaires par période	200
GET	/api/statistiques/mecaniciens	ROLE_USER	Stats par mécanicien (heures facturées, taux)	200

SuiviController

But : Suivi public RDV (client via URL avec token, sans auth).

Méthode	Chemin	Permission	But	Codes
GET	/api/public/suivi/{token}	Public (token)	Statut RDV + photos + estimations	200, 404

ClauseLegaleController

But : Gestion clauses légales (conditions générales, mentions obligatoires facture).

Méthode	Chemin	Permission	But	Codes
GET	/api/clauses-legales	Public	Liste clauses visibles	200
GET	/api/clauses-legales/{code}/active	Public	Retourne version active pour un code	200
POST	/api/clauses-legales	ROLE_ADMIN	Crée clause (versions multiples)	201, 400
PUT	/api/clauses-legales/{id}	ROLE_ADMIN	Update clause	200
POST	/api/clauses-legales/hash	ROLE_USER	Calcule hash des clauses actives (validation signature)	200

AdminAtelierController

But : Création, configuration ateliers (SIRET, TVA intra, horaires, config tarifaires).

Méthode	Chemin	Permission	But	Codes
GET	/api/admin/ateliers	ROLE_SUPER_ADMIN	Liste tous les ateliers	200
POST	/api/admin/ateliers	ROLE_SUPER_ADMIN	Crée atelier + paramètres	201, 400
PUT	/api/admin/ateliers/{id}	ROLE_SUPER_ADMIN	Update config atelier	200

AdminUserProvisioningController

But : Invitation utilisateurs, approbation, rejet (workflow provisioning).

Méthode	Chemin	Permission	But	Codes
GET	/api/admin/users/pending	ROLE_SUPER_ADMIN	Liste invitations en attente	200
POST	/api/admin/users/{id}/approve	ROLE_SUPER_ADMIN	Approuve l'accès utilisateur	200
POST	/api/admin/users/{id}/reject	ROLE_SUPER_ADMIN	Rejette l'accès	200

NotificationController

But : Notifications Mercure (in-app temps réel), emails, SMS (côté utilisateur).

Méthode	Chemin	Permission	But	Codes
GET	/api/notifications	ROLE_USER	Liste notifications utilisateur (paginé)	200
GET	/api/notifications/unread-count	ROLE_USER	Compte notifications non lues	200
POST	/api/notifications/{id}/acknowledge	ROLE_USER	Marque notif comme vue (Mercure)	200
POST	/api/notifications/{id}/mark-read	ROLE_USER	Marque comme lue en DB	200

NotificationProviderController

But : Configuration providers (email, SMS, Mercure), tests, webhooks de retour.

Méthode	Chemin	Permission	But	Codes
GET	/api/admin/notification-providers	ROLE_ADMIN	Liste providers configs	200
POST	/api/admin/notification-providers	ROLE_ADMIN	Crée config (SMTP, Twilio, Mercure)	201, 400
PUT	/api/admin/notification-providers/{id}	ROLE_ADMIN	Update config	200
DELETE	/api/admin/notification-providers/{id}	ROLE_ADMIN	Supprime config	204
POST	/api/admin/notification-providers/{id}/test	ROLE_ADMIN	Envoie test (SMS, email)	200
GET	/api/admin/notification-logs	ROLE_ADMIN	Historique envois	200
GET	/api/admin/notification-templates	ROLE_ADMIN	Templates (RDV confirmation, escalades, etc.)	200
POST	/api/webhooks/notifications/{provider}	Public	Webhook retour Twilio/Mailgun (statuts bounce, ouverture, etc.)	200

CerfaFieldConfigController

But : Personnalisation champs CERFAs PDF (DA, mandat immat, etc.) par atelier.

Méthode	Chemin	Permission	But	Codes
GET	/api/admin/cerfa-config	ROLE_ADMIN	Configuration des champs CERFA pour l'atelier	200

Méthode	Chemin	Permission	But	Codes
PATCH	/api/admin/cerfa-config/{id}	ROLE_ADMIN	Update position/visibility champs	200
POST	/api/admin/cerfa-config/{id}/reset	ROLE_ADMIN	Reset aux defaults	200
GET	/api/admin/cerfa-config/preview/{cerfaRef}	ROLE_ADMIN	Prévisualise le CERFA rendu	200

PublicBookingController

But : Prise de RDV publique (client sans compte).

Méthode	Chemin	Permission	But	Codes
GET	/api/public/ateliers	Public	Liste ateliers acceptant booking public	200
GET	/api/public/prestations-catalogue	Public	Prestations disponibles (query: atelierId)	200
GET	/api/public/slots	Public	Creneaux dispos pour atelierId	200
POST	/api/public/booking	Public	Crée RDV + client auto-crée, retourne token suivi	201, 400

VOPurchaseController

But : Gestion achat véhicule occasion (rachat, stock, revente, marges, facture VO).

Méthode	Chemin	Permission	But	Codes
GET	/api/vo/purchases	ROLE_VO_MANAGER	Liste rachats	200
GET	/api/vo/purchases/{id}	ROLE_VO_MANAGER	Détail rachat	200
GET	/api/vo/purchases/{id}/full	ROLE_VO_MANAGER	Détail enrichi + verdicts vendabilité	200
POST	/api/vo/purchases	ROLE_VO_MANAGER	Crée dossier rachat (OCR CG, PV, LP entry)	201, 400
PATCH	/api/vo/purchases/{id}	ROLE_VO_MANAGER	Update rachat	200
POST	/api/vo/purchases/{id}/confirm	ROLE_VO_MANAGER	Confirme rachat → crée LP + DA SIV	200
POST	/api/vo/purchases/{id}/transition	ROLE_VO_MANAGER	Transition workflow (draft → stock → vente)	200
POST	/api/vo/purchases/{id}/sell	ROLE_VO_MANAGER	Vend rachat → facture VO + rotation LP	201, 400
GET	/api/vo/factures	ROLE_VO_MANAGER	Liste factures VO	200

Méthode	Chemin	Permission	But	Codes
GET	/api/vo/factures/{id}/pdf	ROLE_VO_MANAGER	Télécharge facture VO PDF	200
GET	/api/vo/purchases/{id}/pv-rachat/pdf	ROLE_VO_MANAGER	PV rachat 2 signatures	200
GET	/api/vo/purchases/{id}/cerfa-cession-achat/pdf	ROLE_VO_MANAGER	CERFA cession achat	200
GET	/api/vo/purchases/{id}/da-siv/pdf	ROLE_VO_MANAGER	DA SIV préparée ou finale	200
GET	/api/vo/purchases/{id}/mandat-immat/pdf	ROLE_VO_MANAGER	Mandat immatriculation (CERFA 13757)	200
POST	/api/vo/purchases/{id}/siv/prepare	ROLE_VO_MANAGER	Prépare DA SIV (validation docs + warnings)	200
POST	/api/vo/margin/calculate	ROLE_VO_MANAGER	Calcule marge TVA/normal	200
POST	/api/vo/margin/simulate	ROLE_VO_MANAGER	Simule marge (scénarios achat/vente)	200

VODepotController

But : Gestion dépôt-vente (mandat, commission, reversal).

Méthode	Chemin	Permission	But	Codes
GET	/api/vo/depots	ROLE_VO_MANAGER	Liste dépôts-vente	200
GET	/api/vo/depots/{id}	ROLE_VO_MANAGER	Détail dépôt	200
GET	/api/vo/depots/{id}/full	ROLE_VO_MANAGER	Détail enrichi + verdicts	200
POST	/api/vo/depots	ROLE_VO_MANAGER	Crée mandat dépôt-vente (90j, LP entry)	201
PATCH	/api/vo/depots/{id}	ROLE_VO_MANAGER	Update dépôt	200
POST	/api/vo/depots/{id}/sell	ROLE_VO_MANAGER	Vend dépôt → calcule commission, LP rotation	201
GET	/api/vo/depots/{id}/contrat/pdf	ROLE_VO_MANAGER	Contrat dépôt-vente	200
GET	/api/vo/depots/{id}/mandat-immat/pdf	ROLE_VO_MANAGER	Mandat immat pour vente dépôt	200
POST	/api/vo/depots/{id}/restituer	ROLE_VO_MANAGER	Rend le véhicule au déposant	200

VORemiseEnEtatController

But : Gestion remise en état (FRE : travaux + pièces + coûts).

Méthode	Chemin	Permission	But	Codes
GET	/api/vo/remises-en-etat	ROLE_VO_MANAGER	Liste campagnes FRE	200
GET	/api/vo/remises-en-etat/{id}	ROLE_VO_MANAGER	Détail campagne FRE	200
POST	/api/vo/remises-en-etat	ROLE_VO_MANAGER	Crée campagne FRE	201
PATCH	/api/vo/remises-en-etat/{id}	ROLE_VO_MANAGER	Update campagne	200
GET	/api/vo/remises-en-etat/{id}/pdf	ROLE_VO_MANAGER	PDF état des travaux	200

SecurityReportController

But : Collecte rapports CSP (Content Security Policy violations).

Méthode	Chemin	Permission	But	Codes
POST	/api/security/csp-report	Public	Endpoint CSP report-only (collecte violations)	204

StockController

But : Alertes stock pièces (rupture, min atteint).

Méthode	Chemin	Permission	But	Codes
GET	/api/stock/alertes	ROLE_USER	Alertes stock rupture/min pièces	200

☐ SERVICES MÉTIER (39 au total)

AuditService

But : Centralise les logs d'audit pour conformité RGPD et traçabilité.

Méthodes publiques :

- `log(string $action, string $entityType, int $entityId, string $context, ?int $userId = null): void` — Persiste log audit

Dépendances : EntityManager, CurrentUser

RapportInterventionService

But : Gestion du cycle de vie des rapports d'intervention (brouillon, validation, signature, rectification).

Méthodes publiques :

- `findLatestForRdv(RendezVous $rdv): ?RapportIntervention`
 - `getOrCreateDraft(RendezVous $rdv): RapportIntervention`
 - `createDraft(RendezVous $rdv): RapportIntervention`
 - `prefillFromOR(RapportIntervention $rapport, RendezVous $rdv): void`
 - `calculateNextRevision(?int $currentKm): array` — Déduit prochaine révision (kilométrique)
 - `validateCompleteness(RapportIntervention $rapport): array` — Tableau erreurs si incomplet
 - `validateForMecanicienSignature(RapportIntervention $rapport): array`
 - `signByMecanicien(RapportIntervention $rapport, string $signature, int $mecanicienId): void`
 - `signByClient(RapportIntervention $rapport, string $signature, string $ip): void`
 - `rectify(RapportIntervention $original, string $motif, int $userId): RapportIntervention` — Crée copie rectificative
-

OrdreReparationPolicy

But : Logique métier pour OR (permissions, signature, intégrité, rectification).

Méthodes publiques :

- `canEdit(OrdreReparation $or, User $user): bool`
 - `canSign(OrdreReparation $or): bool`
 - `canRectify(OrdreReparation $or, User $user): bool`
 - `canAddComplementaire(OrdreReparation $or, User $user): bool`
 - `buildSnapshot(OrdreReparation $or): array` — Capture l'état pour signature
 - `computeHash(array $snapshot): string` — SHA256 immuabilité
 - `sign(OrdreReparation $or, string $signatureData, Request $request): string` — Retourne hash signé
 - `verifyIntegrity(OrdreReparation $or): bool` — Valide intégrité post-signature
 - `rectify(OrdreReparation $original, User $user, string $motif): OrdreReparation` — Crée rectificatif
-

PricingService

But : Calcul tarifaire prestations (MO, pièces, TVA).

Méthodes publiques :

- `getConfig(int $atelierId): ?ConfigAtelier`
- `calculatePrestationPrice(Prestation $prestation, Vehicule $vehicule, ?int $atelierId = null, string $tauxType = 'standard'): array`
- `calculateMoPrice(int $minutes, string $tauxType = 'standard', ?int $atelierId = null): array` — Retourne HT + TVA
- `applyPieceMargin(string $prixAchatHt, string $categorie = 'standard', ?int $atelierId = null): array` — Applique marge config

VOMarginService

But : Calcul marge VO (TVA régime normal vs régime particulier marge).

Méthodes publiques :

- `calculateMarginVat(string $purchasePrice, string $salePrice, string $vatRate = '20.0'):` array — Régime particulier marge (Art. 297 A CGI)
 - `calculateNormalVat(string $salePriceHt, string $vatRate = '20.0'):` array — TVA normale
 - `calculateDepotVenteCommission(string $salePrice, string $commissionRate):` array — Commission reversement
-

VODocumentService

But : Gestion documents VO (upload, archivage, RGPD purge, verdicts vendabilité, contrôles légaux).

Méthodes publiques :

- `upload(UploadedFile $file, VOPurchase|VODepotVente $record, string $type, User $user):` VODocument
 - `storeRawContent(string $rawPath, VOPurchase|VODepotVente $record, string $type, User $user):` VODocument
 - `archiveGeneratedPdf(VOPurchase|VODepotVente $record, string $type, string $pdfContent, ?User $user = null):` VODocument
 - `getMissingDocuments(VOPurchase $purchase, bool $includeSale = false):` array
 - `getMissingDocumentsDepot(VODepotVente $depot):` array
 - `getPurchaseSivSummary(VOPurchase $purchase):` array
 - `buildPurchaseLegalChecklist(VOPurchase $purchase):` array
 - `buildDepotLegalChecklist(VODepotVente $depot):` array
 - `getPurchaseSaleBlockers(VOPurchase $purchase):` array — Liste blocages vente
 - `buildPurchaseSaleVerdict(VOPurchase $purchase, array $extraBlockers = []):` array — Verdict complet vendabilité
 - `getDepotSaleBlockers(VODepotVente $depot):` array
 - `buildDepotSaleVerdict(VODepotVente $depot, array $extraBlockers = [], bool $companionComplete = true):` array
 - `getPurchaseDossierStatus(VOPurchase $purchase):` string — Enum statut dossier
 - `getDepotDossierStatus(VODepotVente $depot):` string
 - `getExpiredDocuments(?int $atelierId = null):` array — Docs expirés (RGPD)
 - `getAlerts(?int $atelierId = null):` array — Alertes docum manquantes
 - `batchDocumentTypes(array $purchaseIds, array $depotIds):` array — Batch lookup types
 - `purgeExpiredIdentityDocuments():` int — RGPD purge pièces ID (0 ans rétention)
 - `countStoredSensitiveIdentityDocuments(?int $atelierId = null):` int
-

VOLivrePoliceService

But : Gestion Livre de Police (enregistrement achats/ventes immutables).

Méthodes publiques :

- `createEntryForPurchase(VOPurchase $purchase, User $user): VOLivrePolice`
 - `createEntryForDepotVente(VODepotVente $depot, User $user): VOLivrePolice`
 - `recordSale(VOLivrePolice $entry, VOFacture|null $facture, User $user): void` — Enregistre sortie
-

VONumberingService

But : Numérotation factures VO et n° ordre LP (via séquences PostgreSQL).

Méthodes publiques :

- `nextFactureNumber(?int $atelierId, ?\DateTimeInterface $date = null): string`
 - `nextLivrePoliceOrder(?int $atelierId): int`
-

VOCompanionWorkflowService

But : Workflow signature électronique Companion VO (achat + dépôt-vente).

Méthodes publiques :

- `ensureToken(VOPurchase|VODepotVente $record): bool`
 - `getMode(VOPurchase|VODepotVente $record): string` — 'purchase' | 'depot'
 - `getPartyRoleLabel(VOPurchase|VODepotVente $record): string` — "Vendeur" | "Déposant"
 - `getParty(VOPurchase|VODepotVente $record): ?Client`
 - `getRequiredDocuments(VOPurchase|VODepotVente $record): array`
 - `getAdditionalDocumentOptions(VOPurchase|VODepotVente $record): array`
 - `getGeneratedDocuments(VOPurchase|VODepotVente $record): array`
 - `buildSteps(VOPurchase|VODepotVente $record, array $documents): array`
-

VORemiseEnEtatService

But : Campagnes remise en état VO (travaux + pièces + marge live).

Méthodes publiques :

- `getCampaignsForRecord(VOPurchase|VODepotVente $record): array`
- `getActiveCampaignForRecord(VOPurchase|VODepotVente $record): ?VORemiseEnEtat`
- `hasBlockingActiveCampaign(VOPurchase|VODepotVente $record): bool`

- `createCampaignForRecord(VOPurchase|VODepotVente $record, ?User $requestedBy, array $payload = []): VORemiseEnEtat`
 - `normalizeCampaign(VORemiseEnEtat $campaign): array`
 - `normalizeQueueItem(VORemiseEnEtat $campaign): array`
-

GardiennageService

But : Gardiennage RDV oubliés (relances, tarifs, facturations).

Méthodes publiques :

- `peutDeclencher(RendezVous $rdv): bool`
 - `declencher(RendezVous $rdv, int $userId, string $motif): void`
 - `notifierEntreeGardiennage(RendezVous $rdv): void` — Envoie notif client
 - `calculerMontant(RendezVous $rdv, \DateTime $dateRestitution): string` — bcmath
-

NotificationDispatcher

But : Dispatche notifications par canal (email, SMS, Mercure).

Méthodes publiques :

- `send(NotificationMessage $msg): NotificationResult`
 - `sendFromTemplate(int $atelierId, string $templateCode, string $channel, string $recipient, int $relatedEntityId = 0, string $relatedEntityType = '', array $variables = []): NotificationResult`
 - `testProvider(NotificationProviderConfig $config, string $testRecipient): NotificationResult`
-

MercureNotifier

But : Notifications temps réel Mercure (SSE).

Méthodes publiques :

- `publishToAtelier(int $atelierId, Notification $notif): void`
 - `publishAcknowledged(int $atelierId, int $notifId, int $userId): void`
-

NotificationTemplateCatalog

But : Catalogue templates notifications (textes par défaut, variables).

Méthodes publiques :

- `getDefaults(): array` — Templates defaults

- `ensureDefaultsForAtelier(int $atelierId): int` — Crée defaults atelier
-

NotificationMessage & NotificationResult

But : DTOs notification (input + output).

NotificationMessage : `channel`, `atelierId`, `recipient`, `body`, `subject`, `templateCode`, `relatedEntityType`, `relatedEntityId` **NotificationResult** : `success`, `providerMessageId`, `provider`, `errorMessage`

PdfService

But : Génération PDFs (Twig + DomPDF).

Méthodes publiques :

- `generateOrPdf(OrdreReparation $or): string` — Retourne chemin fichier
 - `generateRapportPdf(RapportIntervention $rapport, ?object $essai = null): string`
 - `generateFacturePdf(Facture $facture): string`
 - `generateDevisPdf(Devis $devis): string`
 - `generateVoFacturePdf(VoFacture $facture): string`
 - `generateLivrePolicePdf(array $entries, ?int $atelierId = null): string`
 - `generateContratDepotVentePdf(VoDepotVente $depot): string`
 - `generatePvRachatPdf(VoPurchase $purchase): string`
 - `generateDaSivPreparationPdf(VoPurchase $purchase, array $blockers = []): string`
 - `generateMandatImmatriculationPdf(VoPurchase|VoDepotVente $record, ?Client $buyer = null): string`
 - `generateCerfaCessionAchatPdf(VoPurchase $purchase): string`
 - `generateVoRemiseEnEtatPdf(array $document, ?int $atelierId = null): string`
 - `buildRdvPhotoContext(?RendezVous $rdv): array`
 - `getCachedPdfPath(string $filename): ?string`
-

CerfaOverlayService

But : Overlay champs CERFA sur templates PDF (personnalisation par atelier).

Méthodes publiques :

- `generateDaSivPreparationPdf(VoPurchase $purchase, Atelier $atelier): string`
 - `generateMandatImmatriculationPdf(VoPurchase|VoDepotVente $record, Atelier $atelier, ?Client $buyer = null): string`
 - `generateCerfaCessionAchatPdf(VoPurchase $purchase, Atelier $atelier): string`
-

PhotoService

But : Gestion photos intervention (upload, EXIF, hash, validation).

Méthodes publiques :

- `allowedTypes(): array`
 - `upload(UploadedFile $file, string $type, RendezVous $rdv, ?string $description = null, ?string $annotationJson = null): PhotoIntervention`
 - `computeHash(string $path): string` — SHA256
 - `extractExif(string $path): ?array`
 - `requirePhotosForTransition(string $transition, RendezVous $rdv): array` — Quelles photos requises pour transition
-

SlotService

But : Calcul créneaux disponibles (ponts × horaires × mécaniciens × absences).

Méthodes publiques :

- `getAvailableSlots(int $atelierId, \DateTime $date, int $estimatedMinutes, ?int $mecanicienId = null): array`
 - `getSlotsForDay(int $atelierId, \DateTime $date, int $estimatedMinutes, ?int $mecanicienId = null): array`
-

PrestationCatalogService

But : Catalogue prestations applicables par type véhicule + tarifs.

Méthodes publiques :

- `getApplicablePrestations(Vehicule $vehicule): array`
 - `calculatePrice(Prestation $prestation, Vehicule $vehicule, ?int $atelierId = null): array`
 - `validateAddition(Prestation $prestation, OrdreReparation $or): void`
-

RendezVousWorkflowService

But : Gestion side-effects workflow (notifications, calculs, audit).

Méthodes publiques :

- `recordCancellation(RendezVous $rdv, User $user, string $reason): void`
- `startWorkSession(RendezVous $rdv, ?\DateTimeInterface $startedAt = null): void`
- `finalizeWorkSession(RendezVous $rdv, ?\DateTimeInterface $endedAt = null): void`
- `handleTransitionSideEffects(string $transition, RendezVous $rdv, User $user, array $context = []): void` — Dispatcher principal

MotoCatalogImporter

But : Import/gestion catalogue motos (marque, modèle, specs, OCR CG).

Méthodes publiques :

- `importFromDefaultFile(): array`
 - `importFromXlsx(string $filePath): array`
 - `prepareCatalogRows(array $rows): array`
 - `importPreparedRows(array $rows): array`
-

HistoriqueEntretienService

But : Agrégation historique entretien véhicule (toutes interventions + suggestions prochaines révisions).

Méthodes publiques :

- `buildHistorique(Vehicule $vehicule): array`
 - `generatePdf(Vehicule $vehicule): string`
-

JoursOuvresService

But : Calcul jours ouvrés (calendrier, jours fériés, fermetures atelier).

Méthodes publiques :

- `compterJoursOuvres(\DateTime $debut, \DateTime $fin, int $atelierId): int`
 - `estJourFerie(\DateTime $date): bool`
 - `estJourFerme(\DateTime $date, int $atelierId): bool`
 - `ajouterJoursOuvres(\DateTime $debut, int $jours, int $atelierId): \DateTime`
-

ClauseLegaleVisibilityService

But : Logique visibilité clauses légales (active, version, code).

Méthodes publiques :

- `pickVisibleClauses(array $clauses, bool $activeOnly): array`
 - `pickPreferredClause(array $clauses): ?ClauseLegale`
-

AtelierCatalogBootstrapService

But : Initialisation catalogue prestations par atelier.

Méthodes publiques :

- `ensurePrestationsForAtelier(?int $atelierId): int` — Nombre prestations créées
-

UserMecanicienSyncService

But : Synchronisation User ↔ Mecanicien (rôles métier).

Méthodes publiques :

- `syncForUser(User $user, bool $allowCreate = true): ?Mecanicien`
-

UserRoleMapper

But : Mappage rôles legacy vs RoleMetier.

Méthodes publiques :

- `mapRoleMetierToLegacyRole(RoleMetier $roleMetier): string`
 - `mapLegacyRoleToRoleMetierCode(?string $legacyRole): ?string`
-

BookingAtelierAccessService

But : Permissions accès atelier pour chaque utilisateur.

Méthodes publiques :

- `getAllowedAteliers(User $user): array`
 - `canAccessAtelier(User $user, ?int $atelierId): bool`
 - `resolvePreferredAtelierId(User $user, ?int $requestedAtelierId = null): ?int`
 - `isServiceClient(User $user): bool`
 - `isSuperAdmin(User $user): bool`
-

CurrentAtelierResolver

But : Résout l'atelier courant depuis JWT.

Méthodes publiques :

- `resolveAtelierId(): ?int`
 - `isGlobalScopeRequested(): bool`
-

ConfigEncryptionService

But : Chiffrement config sensible (clés API, tokens) stockée en base.

Méthodes publiques :

- `encrypt(array $config): string`
 - `decrypt(string $encrypted): array`
-

UserArchiveService

But : Archivage utilisateur RGPD.

Méthodes publiques :

- `archive(User $user, string $reason = 'Archive RGPD'): void`
-

AdminConfigValidator

But : Validation config atelier avant persist.

Méthodes publiques :

- `validateConfigPayload(array $configData, array $horaires = []): array` — Tableau erreurs
-

NotificationProviderConfigSanitizer

But : Merge configs providers (mask secrets, validation).

Méthodes publiques :

- `merge(array $existing, array $submitted): array`
-

AbsenceConflictChecker

But : Détecte chevauchement absences mécaniciens.

Méthodes publiques :

- `isDateRangeValid(\DateTimeInterface $start, \DateTimeInterface $end): bool`
 - `hasConflict(\DateTimeInterface $start, \DateTimeInterface $end, array $existingRanges): bool`
-

VOGeneratedDocumentService

But : Archivage documents générés Companion VO.

Méthodes publiques :

- `archiveCompanionDocumentIfReady(VOPurchase|VODepotVente $record, ?User $user = null, bool $prepareSiv = false): bool`

- `archivePurchaseSivPreparation(VOPurchase $purchase, ?User $user = null, bool $markAsInProgress = false): bool`

VORemiseEnEtatDocumentService

But : Documents générés campagne remise en état.

Méthodes publiques :

- `normalizeDocumentState(VORemiseEnEtat $campaign): array`
- `generateLivePdf(VORemiseEnEtat $campaign): string`
- `signCampaignDocument(VORemiseEnEtat $campaign, string $signatureData, ?User $user = null): void`
- `archiveFallbackDocumentIfMissing(VORemiseEnEtat $campaign, ?User $user = null): ?VODocument`
- `getArchivedDocument(VORemiseEnEtat $campaign): ?VODocument`
- `buildCurrentSnapshot(VORemiseEnEtat $campaign): array`
- `computeHash(array $snapshot): string`

☐ MESSAGES & HANDLERS (4 messages + 4 handlers)

Message	Handler	Dispatché par	But
GeneratePdfMessage	GeneratePdfHandler	Controllers (async/queue)	Génère PDF asynchrone (OR, facture, devis, VO docs)
SendRappelMessage	SendRappelHandler	Controllers + Scheduler	Envoie email rappel RDV (confirmation, J-1, J-3)
SendGardiennageRappelMessage	SendGardiennageRappelHandler	Schedule + Relance cron	Relances gardiennage (J+15, J+30, J+45, J+180)
ProcessScheduledRappels	ProcessScheduledRappelsHandler	Scheduler 8:00	Lance batch traitement rappels du jour

☐ SCHEDULER & CRONS

Fichier : [src/Schedule.php](#)

Cron	Commande	But	Fréquence
0 8 *	ProcessScheduledRappels	Process batch rappels	Chaque jour 8:00
0 3 1	app:rgpd-purge --execute	RGPD purge donnees (clients 3a inactifs, devis > 6m)	1er du mois 3:00
*	app:process-notification-escalations	Escalade notif non-lues	Chaque minute
0 4 *	app:purge-identity-documents	RGPD purge pièces ID (0 jours rétention)	Chaque jour 4:00
0 7 *	app:relance-client-stockage	Gardiennage relances (J+15/30/45/180)	Chaque jour 7:00
0 6 *	app:check-da-siv-expiry	Alerte DA SIV J+10 expiration / J+15 expiré	Chaque jour 6:00
30 6 *	app:check-depot-vente-mandat	Alerte mandat dépôt-vente J-7 avant expiration	Chaque jour 6:30
0 9 *	app:rappel-prochaine-revision	Rappel révision véhicule (J-30 prochaine KM)	Chaque jour 9:00

☐ COMMANDES CLI (14 au total)

Commande	Signature	But	Fréquence
app:create-admin	create-admin	Crée super-admin + atelier par défaut (setup initial)	Manuel — une fois
app:seed	seed [--demo]	Seed données ref (rôles, catégories, horaires, config)	Manuel — setup
app:seed-reset	seed-reset	Purge + réinitialise données demo	Manuel — dev
app:rgpd-purge	rgpd-purge [--execute]	Purge données > durée rétention (clients 3a, devis 6m, audit 3a, IDs 0j)	Cron 1er/mois 3:00 --execute
app:process-notification-escalations	process-notification-escalations	Escalade notifs non-lues (T+5/10/30 min)	Cron chaque minute**
app:purge-identity-documents	purge-identity-documents	RGPD purge pièces ID stockées (rétention = 0j)	Cron chaque jour 4:00
app:check-no-show	check-no-show	Auto-transition RDV no-show si heure+30min < now	Cron? Manuel?

Commande	Signature	But	Fréquence
app:relance-client-stockage	relance-client-stockage	Envoie relances gardiennage (J+15/30/45/180 jours ouvrés)	Cron chaque jour 7:00
app:check-da-siv-expiry	check-da-siv-expiry	Alerte DA SIV J+10 / expiration J+15	Cron chaque jour 6:00
app:check-depot-vente-mandat	check-depot-vente-mandat	Alerte mandat dépôt-vente J-7 avant expiration	Cron chaque jour 6:30
app:import-moto-catalog	import-moto-catalog [file.xlsx]	Import/update catalogue motos (marque, modèle, specs)	Manuel + cron?
app:rappel-prochaine-revision	rappel-prochaine-revision	Rappel client révision J-30 avant prochaine date estimée	Cron chaque jour 9:00
app:send-rappels	send-rappels [type]	Envoie rappels RDV en batch (confirmation, J-1, J-3)	Manuel / Cron via Messenger?
app:backfill-snapshots	backfill-snapshots	Remplit colonnes snap_* entités (migration data)	Manuel — une fois

▣ RÉSUMÉ DENSITÉ & COMPLEXITÉ

Couche	Fichiers	Lignes approx	Complexité
Controllers	43	~8000	Haute (multi-endpoints, validations, workflows)
Services	39	~12000	Très haute (logique métier, RGPD, VO, notifications)
Messages + Handlers	8	~800	Moyenne (async simple)
Commands	14	~1500	Moyenne (crons, batch)
TOTAL	104	~22000	Très haute

▣ SÉCURITÉ & PERMISSIONS

Hiérarchie des rôles :

- ROLE_SUPER_ADMIN — Bypass total TenantFilter, accès cross-ateliers, audit log obligatoire
- ROLE_ADMIN — Admin atelier (config, users, archives)
- ROLE_VO_MANAGER — Gestion VO (rachats, dépôts, marges)
- ROLE_USER — Accès général
- RoleMétier** — Rôles paramétrables (permissions granulaires par module/action)

Authentification : JWT + Refresh Token (révocation via RevokedToken)

Multi-tenant : TenantFilter auto-appliqué sur toutes requêtes (colonne atelierId implicite)

⚠ POINTS CRITIQUES

1. **OR signé = immuable** — Modification seulement via rectificatif (audit + bloc workflow)
 2. **Essai routier obligatoire** — Bloquant avant terminer RDV
 3. **DA SIV bloquant** — Sans DA enregistrée, VO invendable
 4. **RGPD purge** — Pièces ID, audit IPs, clients 3a inactifs (automatisé cron)
 5. **TVA marge vs normale** — Régimes exclusifs facture VO (Art. 297 A vs 256 CGI)
 6. **Livre Police immuable** — Pas PUT/DELETE après création
 7. **Notification escalade** — Chaque minute, relances T+5/10/30 min auto
-

FIN DU RAPPORT.

Maintenant que j'ai collecté toutes les informations, je vais produire le rapport exhaustif :

Rapport Exhaustif du Frontend Nuxt 3 — AtelierMBZ

☐ Table des matières

1. [Configuration Nuxt](#)
 2. [Pages](#)
 3. [Composables](#)
 4. [Stores Pinia](#)
 5. [Composants](#)
 6. [Middlewares](#)
 7. [Layouts](#)
 8. [Plugins](#)
 9. [Tests Vitest](#)
-

Configuration Nuxt

`nuxt.config.ts`

Modules activés :

- `@nuxt/ui` — Système de design UI
- `@pinia/nuxt` — Gestion d'état

SSR : `false` (SPA)

Runtime Config public :

- `apiBase` : `/api`
- `mercureUrl` : `/.well-known/mercure`

Colormode : Dark mode par défaut

Nitro Route Rules :

- Proxy API : `/api/` → `http://localhost:8000/api/`
 - CSP Report-Only sur toutes les pages (observer mode)
 - Whitelist : `fonts.googleapis.com`, `Mercure`
 - Headers de sécurité : `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`
-

Pages

Vue d'ensemble par URL

URL	Métier	But	État
/	Resp. atelier, Admin	Dashboard KPI : CA, RDV, panier moyen, occupation ponts, charge, statuts	△ Implémenté
/login	Public	Authentification Google SSO + fallback local	△ Implémenté
/profile	Tous	Profil utilisateur : identité, rôle, permissions, atelier actif	△ Implémenté
/planning	Réceptionniste	Planning drag & drop ponts/mécaniciens, création RDV rapide, filtrage	△ Implémenté
/workshop	Resp. atelier	Pilotage ponts, mécaniciens, alertes, gardiennage, absences	△ Implémenté
/mecanicien	Mécanicien	Espace mobile-first : interventions en cours, chrono, checkup, essai routier, photos, travaux complémentaires	△ Implémenté
/rdv/new	Réceptionniste	Prise de RDV : client, véhicule, type intervention, pont, mécanicien, temps estimé	△ Implémenté
/rdv/[id]	Tous	Détail RDV (redirection conditionnelle)	△ Implémenté
/rdv/index	Réceptionniste	Liste RDV filtrée (redirection vers /planning ou /rdv/new selon rôle)	△ Implémenté
/ordres/index	Tous	Liste des OR : client, véhicule, montant, statut ; filtres par statut	△ Implémenté
/ordres/[id]	Tous	Détail OR : création/consultation, lignes, signatures, rectification	△ Implémenté
/rapport/[rdvId]	Mécanicien + Réceptionniste	Rapport d'intervention : travaux, km restitution, essai routier, signatures, rectification	△ Implémenté
/devis/index	Tous	Liste des devis : client, véhicule, montant, statut	△ Implémenté
/devis/[id]	Tous	Détail devis : lignes, totaux, validité, signature client	△ Implémenté
/clients/index	Réceptionniste	Liste clients : recherche, stats (avec RDV, véhicules), CA total	△ Implémenté

URL	Métier	But	État
/clients/[id]	Réceptionniste	Détail client : contact, véhicules associés, historique RDV	△ Implémenté
/motos	Tous	Catalogue moto : recherche, filtre catégorie, détails techniques	△ Implémenté
/suivi	Réceptionniste	Suivi live interventions par mécanicien (temps réel Mercure)	△ Implémenté
/tarifs	Admin	Configuration grilles tarifaires : prestations, taux MO, marges pièces	△ Implémenté
/facturation/index	Comptable + Admin	Liste factures (EN RÉÉCRITURE — NE PAS TOUCHER)	□ Réécriture
/stock/index	Gestionnaire stock	Gestion pièces détachées (EN RÉÉCRITURE — NE PAS TOUCHER)	□ Réécriture
/admin/index	Admin	Hub admin : accès ateliers, users, config, absences, ponts, etc.	△ Implémenté
/admin/ateliers	Super-admin	CRUD ateliers (créer, lister, configurer)	△ Implémenté
/admin/users	Admin	CRUD utilisateurs : rôles, permissions, SSO	△ Implémenté
/admin/roles	Super-admin	Gestion rôles globaux (CRUD)	△ Implémenté
/admin/roles-metier/index	Super-admin	Lister rôles métier (CRUD)	△ Implémenté
/admin/roles-metier/[id]	Super-admin	Détail rôle métier : permissions granulaires	△ Implémenté
/admin/prestations	Admin	CRUD prestations (tarification par prestation)	△ Implémenté
/admin/ponts	Admin	CRUD ponts : équipements, affectations mécaniciens	△ Implémenté
/admin/config	Admin	Configuration atelier : identité, tarifs, horaires, gardiennage, modules VO	△ Implémenté
/admin/absences	Admin	CRUD absences mécaniciens (congé, maladie, formation)	△ Implémenté
/admin/audit	Super-admin	Journal d'audit global (AuditLog cross-atelier)	△ Implémenté
/admin/clauses-legales	Admin	Gestion clauses légales par prestation	△ Implémenté

URL	Métier	But	État
/admin/templates-documents	Super-admin	Édition templates email/SMS	△ Implémenté
/admin/demandes-travaux-supp	Admin	Configuration demandes complémentaires (workflow, escalade)	△ Implémenté
/admin/notifications/providers	Admin	Configuration providers (SMTP, Twilio, Mercure)	△ Implémenté
/admin/cerfa-config	Super-admin	Paramètres CERFA / SIV (numérotation, données)	△ Implémenté
/vo/index	Gestionnaire VO	Dashboard VO : stock prioritaire, alertes, stats	△ Implémenté
/vo/rachats/new	Gestionnaire VO	Wizard création rachat : vendeur → véhicule → documents → FRE → confirmation	△ Implémenté
/vo/rachats/index	Gestionnaire VO	Lister rachats avec filtres (statut, marge, jours stock)	△ Implémenté
/vo/rachats/[id]	Gestionnaire VO	Détail rachat : véhicule, documents obligatoires, Livre Police, verdict vendabilité	△ Implémenté
/vo/depots/new	Gestionnaire VO	Wizard création dépôt-vente : déposant → véhicule → mandat → confirmation	△ Implémenté
/vo/depots/index	Gestionnaire VO	Lister dépôts avec filtres (statut, mandat, net déposant)	△ Implémenté
/vo/depots/[id]	Gestionnaire VO	Détail dépôt-vente : véhicule, mandat, revenus, LP	△ Implémenté
/vo/remises-en-etat/index	Gestionnaire VO	File atelier FRE : prestations, pièces, coûts, marge temps réel	△ Implémenté
/vo/factures	Comptable VO	Lister factures VO (régime TVA marge ou normal)	△ Implémenté
/vo/livre-police	Gestionnaire VO	Registre officiel acquisitions/ventes (immuable)	△ Implémenté
/vo/documents	Admin VO	Gestion documents VO (uploads, RETENTION_YEARS)	△ Implémenté
/public/booking	Client public	Prise de RDV en ligne : sélection atelier, créneaux, confirmation	△ Implémenté
/public/suivi/[token]	Client public	Suivi RDV avec code token (progress steps)	△ Implémenté
/public/suivi	Client public	Recherche suivi (saisir code token)	△ Implémenté

URL	Métier	But	État
/public/companion/[token]	Client public	Signature OR + essai routier + restitution (PDA)	△ Implémenté
/public/vo-companion/[token]	Vendeur / Déposant VO	Signature documents VO (PDA) : pièce identité, CG, etc.	△ Implémenté
/public/demande/[token]	Client public	Validation demande travaux complémentaires (avec signature)	△ Implémenté
/public/mentions-legales	Public	Mentions légales	△ Implémenté
/public/politique-confidentialite	Public	Politique de confidentialité	△ Implémenté

Pages complexes — Détails

/planning.vue

Métier cible : Réceptionniste

Sections de l'écran :

1. **KPI Bar** : Charge visible (MO + pièces), conflits, non-assignés, retards
2. **Légende de couleurs** : À valider, réservé, confirmé, en cours, terminé
3. **Horaires affichage** : Range horaire atelier + jours ouverture
4. **Filtre mécaniciens** : Toggle par mécano avec couleur
5. **View toggle** : Grid (drag-drop) vs List (tableau)
6. **PlanningGrid** : Composant affichage visuellement slots par pont × temps
7. **PlanningList** : Alternative tabulaire des RDV
8. **Quick Create Modal** : Création RDV rapide depuis planning

Stores utilisés : useRdvStore, useAtelierStore, useAuthStore

Composables utilisés : useApi, useAuth, useFormat

Endpoints API clés :

- GET /rendez-vous (filtre date, statut, mécanicien, pont)
- PATCH /rendez-vous/{id} (déplacement, transition)
- GET /mecaniciens (liste mécaniciens pour filtres)
- GET /ponts (liste ponts)
- GET /horaires-atelier (horaires)

/mecanicien.vue

Métier cible : Mécanicien (mobile-first, tablette)

Sections :

1. **Header** : Avatar, date, KPIs (en cours, à faire, terminés, % journée)
2. **Priority Action** : Prochaine action (basée sur statut RDV)
3. **Active Intervention Card** :
 - Nav tabs : Intervention, Check-up, Essai routier, Travaux, Photos, Notes
 - **Intervention** : Client, véhicule, pont, motif, contexte réception (OR signé, km, carburant, priorité)
 - **Chrono** : Elapsed / total (transition `start_travail`, `mettre_en_pause`, `reprendre`)
 - **Check-up** : Grille d'inspection des points (toggle OK/NOK/N/A), photos associées
 - **Essai routier** : Km début/fin, points de contrôle, anomalies, signature mécanicien
 - **Travaux complémentaires** : Liste demandes en escalade, accept/reject
 - **Photos** : Galerie photos intervention (types : `en_cours`, `apres_travaux`, `restitution`, `probleme`)
 - **Notes** : Notes texte mécanicien

Stores utilisés : `useRdvStore`, `useAuthStore`

Composables utilisés : `useApi`, `useFormat`, `useNotifications` (Mercure)

Composants enfants :

- `MecanicienNav` — Onglets section
- `MecanicienSection` — Container section repliable
- `MecanicienCheckupGrid` — Grille checkup interactive
- `MecanicienChrono` — Affichage chrono temps

Endpoints API clés :

- `GET /rendez-vous/{id}/mecanicien` (contexte mécanicien)
- `GET /essai-routier/{rdvId}` (essai routier)
- `PATCH /essai-routier/{id}` (sauvegarder essai)
- `PATCH /rendez-vous/{id}` (transitions : `start_travail`, `mettre_en_pause`, etc.)
- `POST /photos/upload` (upload photos intervention)
- `GET /rapport-intervention/{rdvId}` (rapport)

`/workshop.vue`

Métier cible : Responsable atelier

Sections :

1. **KPI Bar** : Occupation ponts %, RDV aujourd'hui, mécaniciens actifs, conflits
2. **Tabs** :
 - **Ponts** : Vue ponts (grid) ou liste. Alert badges (CG manquante, CT expiré, OR non signé, essai manquant, gardiennage). Véhicule timeline par pont.
 - **Mécaniciens** : Charge par mécanicien, historique journée
 - **Alertes** : Listing alertes filtrables (docs manquants, expirations, workflow bloqués)

- **Gardiennage** : Motos en gardiennage (relances, abandon)
- **Absences** : Absences mécaniciens vue calendrier

Stores utilisés : `useRdvStore`, `useAuthStore`, `useAtelierStore`

Composables utilisés : `useApi`, `useNotifications` (Mercure)

Composants enfants :

- `WorkshopAlertBadges` — Filtres d'alerte
- `WorkshopQuickActions` — Actions rapides
- `WorkshopVehicleTimeline` — Timeline véhicule par pont
- `WorkshopSkeletonBays` — Skeleton loading

`/vo/rachats/new.vue`

Métier cible : Gestionnaire VO

Wizard 4 étapes :

1. **Vendeur** : Recherche client ou création rapide (prénom, nom, tél, email, adresse)
2. **Véhicule** : OCR carte grise ou saisie manuelle (plaque, VIN, marque, modèle, MEC, cylindrée, couleur)
3. **Documents** : Upload documents obligatoires (CG, CT, non-gage, PV rachat, PV d'état)
4. **FRE** : Frais remise en état (lignes, pièces, coûts, marge calculée en temps réel via `VOMarginService`)

Secteur Companion : QR code immédiat pour ouverture `/public/vo-companion/{token}` (PDA).

Stores utilisés : `useVoStore`, `useAuthStore`

Composables utilisés : `useApi`, `useCarteGriseOcr`, `useQrCode`, `usePdfDownload`, `voVehicleForm`

Endpoints API clés :

- `POST /vo/rachats` (création brouillon)
- `PATCH /vo/rachats/{id}` (mise à jour étapes)
- `POST /vo/rachats/{id}/companion` (génération token Companion PDA)
- `POST /ocr/carte-grise` (OCR carte grise)

`/public/companion/[token].vue`

Métier cible : Client final (signature OR + essai + restitution)

Sections :

1. **RDV Info** : Récapitulatif client, véhicule, statut
2. **Réception PDA** (statut == `reception`) : Signature client sur OR
3. **Intervention terminée** (statut == `termine`) : Signature client rapport d'intervention + essai routier
4. **Status Pills** : Indicateurs photos, OCR CG, signatures

Composants :

- Canvas signature bidirectionnelle (pointerdown/pointermove/pointerup)
- Affichage rapport d'intervention texte

Endpoints API clés :

- GET /public/companion/{token} (validation token)
- GET /rendez-vous/{id}/public (données RDV avec token)
- PATCH /rendez-vous/{id}/signature-or (signature OR client)
- PATCH /rendez-vous/{id}/signature-rapport (signature rapport client)

/public/vo-companion/[token].vue

Métier cible : Vendeur / Déposant VO (signature documents VO)

Sections :

1. **Stepper** : Étapes étapes wizard (Vendeur/Déposant → Véhicule → Documents → Signature)
2. **Seller** : Transcription pièce identité (type, numéro, date) — **pas d'upload**
3. **Vehicule** : Saisie véhicule (OCR CG ou manual)
4. **Documents** : Liste documents générés (PV rachat, contrat dépôt, CERFA, etc.)
5. **Signature** : Canvas électronique

Stores utilisés : useVoStore

Composables utilisés : useApi, voCompanionDraftSync, voVehicleForm, usePdfDownload

/vo/remises-en-etat/index.vue

Métier cible : Gestionnaire VO

Contenu :

- Liste véhicules VO en FRE
- Chaque véhicule : marque/modèle, plaque, prestations + pièces associées
- Colonnes : désignation, coût HT, quantité, temps, statut (proposée, validée, facturée)
- **VORemiseEnEtatCard** composant enfant : affiche ligne + pièces, marge temps réel via VOMarginService

Stores utilisés : useVoStore

Composables utilisés : useApi

Composants enfants :

- VORemiseEnEtatCard — Carte FRE ligne + pièces
-
-

Composables

Composable	Fonctions exportées	But
useApi	<code>\$fetch<T></code> , <code>get<T></code> , <code>post<T></code> , <code>patch<T></code> , <code>put<T></code> , <code>delete<T></code> , <code>upload</code>	Client HTTP universel avec auth, refresh token auto, gestion erreurs
useAuth	<code>login()</code> , <code>getGoogleLoginConfig()</code> , <code>startGoogleLogin()</code> , <code>exchangeGoogleCode()</code> , <code>logout()</code> , <code>fetchMe()</code> , <code>hasSection()</code> , <code>hasPerm()</code> , <code>hasStatsAccess()</code>	Authentification SSO Google + local, récupération user, permissions
useFormat	<code>formatDate()</code> , <code>formatTime()</code> , <code>formatCurrency()</code> , <code>formatDuration()</code>	Formatage dates, heure, devise EUR, durée en H:MM
useValidation	<code>isValidPhone()</code> , <code>isValidEmail()</code> , <code>isValidPlate()</code> , <code>validateClientFields()</code>	Validation regex champs français (tél 10 chiffres, plaque AA-123-AA, email)
useNotifications	<code>fetchNotifications()</code> , <code>fetchUnreadCount()</code> , <code>markAsRead()</code> , <code>subscribeToMercure()</code>	Notifications via EventSource Mercure ou poll (websocket temps réel)
useQrCode	<code>useQrCode(text, size) → { dataUrl }</code> , <code>generateQrDataUrl()</code>	Génération QR codes localement (zéro service externe)
usePdfDownload	<code>downloadPdf(path, filename)</code> , <code>openPdf(path)</code>	Téléchargement/aperçu PDF via fetch blob (inclut credentials)
useMotoAutocomplete	<code>searchMoto()</code> , <code>getMotoDetails()</code>	Autocomplete marque/modèle moto via API
useDebounceFn	<code>debounceFn(fn, delay)</code>	Debounce fonction
useCarteGriseOcr	<code>startOcr()</code> , <code>pickOcrImageFile()</code> , <code>buildOcrComparison()</code> , <code>vinErrorMessage()</code> , <code>isValidVin()</code>	OCR carte grise (téléchargement image + traitement backend), validation VIN ISO 3779
useVoHelpers	<code>formatPrice()</code> , <code>computeMargin()</code>	Helpers VO (calculs prix/marge)
voVehicleForm	<code>isValidVin()</code> , <code>vinErrorMessage()</code> , <code>extractVehicleCategoryId()</code> , <code>applyVehicleToForm()</code> , <code>buildVoVehiclePayload()</code>	Gestion formulaire véhicule VO (VIN validation, payload API)
voCompanionDraftSync	<code>syncDraftField()</code> , <code>syncDraftBoolean()</code> , <code>adoptDraftEntity()</code>	Synchronisation brouillon Companion PDA ↔ saisie comptoir (conflit détection)

Composable	Fonctions exportées	But
voRefurbishmentCard	<code>selectRefurbishmentCampaignId()</code> , <code>buildRefurbishmentLineForms()</code> , <code>buildRefurbishmentPieceForms()</code> , <code>toRefurbishmentDateTimeLocal()</code>	Gestion formulaires FRE (campagnes, lignes, pièces)

Détails composables clés

useApi

Singleton refresh token : Si 401 → refresh auto (serialisé pour éviter race conditions parallèles).

Gestion FormData : Auto-détection `Content-Type` .

Logging : Erreurs en console si verbeux.

Fallback local cert : Recommandation si HTTPS localhost échoue.

useAuth

Contexte : Multi-ateliers, rôles métier, permissions granulaires.

Refresh stratégie :

- `silentRefresh()` → POST `/auth/refresh` (include credentials)
- `fetchMe()` → GET `/auth/me` pour rechargement contexte

Permissions :

- `hasSection(section)` : Vérif dans `role_permissions.sections_json`
- `hasPerm(module, action)` : Vérif dans `role_permissions.permissions_json`
- `hasStatsAccess()` : Accès dashboard (resp. atelier min)

useNotifications

Transport : Priorité Mercure (EventSource) → fallback poll (setInterval).

API endpoints :

- GET `/notifications?status=unread&limit=50` (fetch)
- GET `/notifications/unread-count` (count badge)
- PATCH `/notifications/{id}/read` (marquer lue)

useCarteGrise0cr

Flux :

1. Utilisateur sélectionne fichier image (ou PDF ignoré)
2. `start0cr(file)` → POST `/ocr/carte-grise` (multipart)

3. Retour JSON : plaque, marque, modèle, VIN, annee, cylindree, etc.

4. Comparaison vs saisie manuelle : `build0crComparison()` retourne { tone, message, canUseBase }

Validation VIN ISO 3779 :

- 17 caractères exactement
- Pas I, O, Q (ambigu visuellement)
- Alphanumérique

Stores Pinia

Store	State	Getters	Actions	But
app	<code>sidebarOpen</code> , <code>loading</code> , <code>currentSection</code>	—	<code>toggleSidebar()</code> , <code>setSection()</code> , <code>setLoading()</code>	UI globale (sidebar, section active)
auth	<code>user: UserData</code>	<code>isAuthenticated</code> , <code>fullName</code> , <code>role</code> , <code>atelierId</code> , <code>atelierNom</code>	<code>setUser()</code> , <code>clearUser()</code>	Contexte user (JWT payload, permissions)
atelier	<code>modules</code> , <code>branding</code> , <code>loaded</code>	<code>isModuleEnabled()</code>	<code>setModules()</code> , <code>setBranding()</code> , <code>setConfig()</code> , <code>clearModules()</code>	Modules VO/atelier activés, branding (logo, nom)
rdv	<code>rdvs</code> , <code>currentRdv</code> , <code>filters</code> , <code>loading</code> , <code>planningDate</code>	—	<code>fetchRdvs()</code> , <code>getRdvById()</code> , <code>updateRdv()</code> , <code>createRdv()</code>	Gestion RDV en mémoire (cache planning)
billing	<code>factures</code> , <code>currentFacture</code> , <code>loading</code>	—	<code>fetchFactures()</code> , <code>createFacture()</code> , <code>addPayment()</code> , <code>createAvoir()</code> , <code>downloadPdf()</code>	Gestion factures atelier (archive, téléchargement PDF)
stock	<code>pieces</code> , <code>loading</code>	<code>alertes</code> (pièces < seuil)	<code>fetchPieces()</code> , <code>createPiece()</code> , <code>updatePiece()</code>	Gestion stock pièces détachées
vo	<code>purchases</code> , <code>depots</code> , <code>refurbishments</code> , <code>loading</code>	—	<code>fetchPurchases()</code> , <code>createPurchase()</code> , <code>updatePurchase()</code> , <code>fetchDepots()</code> , <code>createDepot()</code> , <code>updateDepot()</code> , etc.	Gestion dossiers VO (rachats, dépôts, FRE)

Détails stores clés

useAuthStore

```
interface UserData {
  id: number
  email: string
  username: string
  nom?: string | null
  prenom?: string | null
  role: string // Legacy
  roles?: string[] // New (ROLE_*, ROLE_SUPER_ADMIN)
  atelier_id?: number | null
  atelier_nom?: string | null
  auth_provider?: string
  access_status?: string
  is_pending_validation?: boolean
  needs_atelier_assignment?: boolean
  role_permissions?: {
    sections_json: string[]
    permissions_json: string[]
  } | null
  role_metier?: {
    id: number
    code: string
    libelle: string
    base_role?: string
    permissions?: Array<{ module: string; action: string; scope: string }>
  } | null
}
```

useAtelierStore

Feature modules par défaut :

```
{
  dashboard: true,
  rdv: true,
  planning: true,
  workshop: true,
  mecanicien: true,
  suivi: true,
  clients: true,
  or: true,
  devis: true,
  facturation: true,
```

```
stock: true,
absences: true,
admin: true,
tarifs: true,
vo: false, // Disabled par défaut
}
```

Branding : Logo atelier, nom, adresse, SIRET, TVA intracom

`useRdvStore`

Normalisation API Platform : Aplati champs imbriqués (client.nom → client_nom, etc.) pour alignement templates.

Payload update :

```
{
  date_rdv,
  heure_rdv,
  type_intervention,
  commentaire,
  temps_estime,
  prix_estime,
  prix_final,
  kilometrage,
  statut,
  client: IRI,
  vehicule: IRI,
  pont: IRI,
  mecanicien: IRI
}
```

`useVoStore`

Actions principales :

- `fetchPurchases(limit=200)` → GET `/vo/rachats?itemsPerPage=200`
 - `createPurchase(payload)` → POST `/vo/rachats`
 - `updatePurchase(id, patch)` → PATCH `/vo/rachats/{id}`
 - `fetchDepots()`, `createDepot()`, `updateDepot()` (dépôts-vente)
 - `fetchRefurbishments(depotId)` → GET `/vo/remises-en-etat?depot={id}`
 - `createRefurbishment(depotId)` → POST `/vo/depots/{depotId}/remise-en-etat`
 - `fetchLivrePolicies()` → GET `/vo/livre-police` (immuable, GET only)
-
-

Composants

Composant	Props	Événements	But
AppPageHeader	title, subtitle, backTo	—	En-tête page uniformisé (titre, sous-titre, actions slot)
AppBanner	variant (info/warning/danger), title, description, dismissible	@dismiss	Bannau d'alerte/notification en haut page
AppBreadcrumbs	—	—	Fil d'Ariane navigation
AppEmptyState	icon, title, description, actionLabel	@action	État vide (pas de données)
AppErrorState	title, description	@retry	État erreur (échec chargement API)
AppLoadingState	title, description	—	État loading (skeleton ou spinner)
AppModal	v-model:open, size (lg/md/sm), title	—	Modal générique avec slot content
AppNotificationBell	unreadCount	@click	Cloche notifications header
AppPageHeader	title, subtitle	—	En-tête page (utilisé dans planning, mecanicien, etc.)
AppPdfEmbed	src (URL PDF)	—	Afficheur PDF inline
AppPeriodSelector	v-model, presets, selectedPreset	@preset, @update:model-value, @refresh	Sélecteur période (dashboard)
AppQuickActions	—	—	Raccourcis actions rapides
AppSkeletonCard	lines (nombre de lignes)	—	Skeleton loading card
AppSkeletonKpi	—	—	Skeleton loading KPI
AppSkeletonTable	rows, cols	—	Skeleton loading tableau
StatusBadge	status (statut RDV/OR)	—	Badge couleur statut (réservé, confirmé, terminé, etc.)
SidebarLink	to, icon, label, section, badgeCount	—	Lien sidebar actif selon route

Composant	Props	Événements	But
StatsCard	title, value, icon, color	—	Carte stat KPI
UTable	data, columns, loading	—	Tableau sortable Nuxt UI
MecanicienNav	v-model, sections (tabs)	@update:model-value	Onglets mécanicien (Intervention, Essai, etc.)
MecanicienSection	title, sectionKey, icon, defaultOpen	—	Section repliable écran mécanicien
MecanicienCheckupGrid	checkup, items (liste points), photos	@toggle	Grille checkup 3-états (OK/NOK/N/A)
MecanicienChrono	elapsed, total (secondes), status	—	Affichage chrono interactif
PlanningGrid	ponts, rdvs, horaires, mecaniciens, canDrag	@select-rdv, @move-rdv, @create-at, @dates-changed	Grille drag-drop planning ponts x temps
PlanningList	rdvs, mecaniciens, canCreate	@select-rdv, @action-transition, @create-request	Vue list alternative planning
NotificationPopIn	—	—	Pop-in notifications temps réel (Mercure)
VOCompanionCard	companion, generatedDocuments	—	Carte Companion VO (QR + docs)
VODossierMotoCard	mode (purchase/depot), dossierId, vehicule, documents, missingDocuments, legalChecklist, saleVerdict	@reload-detail	Vue documents obligatoires VO
VONav	—	—	Nav VO sections (Rachats, Dépôts, Remises, etc.)
VORemiseEnEtatCard	campaign, lines, pieces, margin	—	Carte ligne FRE avec calcul marge
WorkshopAlertBadges	v-model, counts	@update:model-value	Filtres badges alertes workshop
WorkshopQuickActions	—	—	Actions rapides workshop (créer RDV, etc.)
WorkshopSkeletonBays	—	—	Skeleton ponts

Composant	Props	Événements	But
WorkshopVehicleTimeline	rdvs	—	Timeline véhicule par pont

Middlewares

Middleware	Type	But	Comportement
auth.global	Global (avec global suffix)	Authentification + autorisation	Routes publiques : /login, /public/, /companion/. Routes privées : fetch user, vérif rôle, vérif access sections (admin/audit), fallback home si pas accès dashboard, chargement config atelier. Refresh auth J+5min si stale
local-protocol	Global	Support protocole personnalisé atelier://	Peut être interceptor deeplinks

Layouts

Layout	Utilisation	Contenu
default	Pages authentifiées (planning, mecanicien, admin, etc.)	Sidebar (logo atelier, menu groupé, profil/logout en bas), Main area (topbar avec breadcrumbs/actions, main content), AppNotificationBell
public	Pages publiques (login, booking, companion, suivi, mentions légales)	Fond gradient obscur, footer liens légaux, pas de sidebar

Plugins

Plugin	Type	But
auth-heartbeat.client	Client-side	Polling activité (pointerdown, keydown, scroll, touchstart). Refresh session auto toutes les 5min si actif + dans fenêtre visible.

Tests Vitest

Tests Frontend Existants (19 tests)

Test file	Suites	Tests	Objectif
useApi.test.ts	1 suite <code>useApi</code> <code>auth</code> <code>handling</code>	5+ tests	Gestion 401 auto-refresh, retry après refresh, composant erreurs
useCarteGriseOcr.test.ts	1 suite	2+ tests	Sélection image OCR (filtre PDF), warning si PDF seul
voCompanionDraftSync.test.ts	1 suite	4+ tests	Sync champs brouillon (respect édits manuels, booleans), adoption entités sans conflit
voRefurbishmentCard.test.ts	1 suite	3+ tests	Sélection campagne FRE, construction formulaires lignes/pièces, format datetime-local
voVehicleForm.test.ts	1 suite	2+ tests	Validation VIN ISO 3779 (17 chars, pas I/O/Q), extraction category ID, build payload
voStore.test.ts	1 suite <code>useVoStore</code>	3+ tests	Création dépôt draft, PATCH finalization, endpoints FRE

Commandes de test

```
npm run test # Vitest watch mode
npm run test:coverage # Couverture
```

Architecture Générale

Flux d'authentification

1. **Login page** : Form email/pw ou SSO Google
2. `useAuth().login()` / `startGoogleLogin()` → POST `/auth/login` ou redirect Google OAuth
3. Response contient `user` + JWT stocké en cookie (include credentials)
4. `auth.global.ts` middleware : `fetchMe()` peuple `useAuthStore`
5. Polling refresh auth chaque 5 min si stale (plugin `auth-heartbeat.client`)

Flux autorisation

- Checker `useAuth().hasSection('rdv')` avant d'afficher `/rdv`

- Vérif `hasPerm('facture.delete')` côté UI + côté API

Flux notifications temps réel

- Connexion Mercure via `useNotifications().subscribeToMercure()`
- EventSource `/.well-known/mercure?topics=...` (souscription sélective par atelier, rôle)
- Fallback polling si WebSocket indisponible

Flux VO (Rachat, Dépôt, FRE)

```
new.vue (wizard 4 étapes)
↓
createPurchase / createDepot (POST /vo/rachats ou /vo/depots)
↓
[QR code Companion PDA généré]
↓
/public/vo-companion/[token]
  (vendeur signe documents via PDA)
↓
rapport détail /vo/rachats/[id]
  (voir LP, verdict vendabilité, docs manquants)
↓
FRE via remises-en-etat/index
  (lignes prestations + pièces, marge recalculée)
↓
Vente / emission facture VO
```

Flux Documents

- `useCarteGrise0cr` : OCR CG en local (tesseract.js) + API backend fallback
- `usePdfDownload` : Fetch PDF avec credentials inclus (pas window.open simple)
- `voRefurbishmentCard` : Gestion formulaires FRE dynamiques

Conventions de code

- **Script setup** : Tous les `.vue` utilisent `<script setup lang="ts">`
- **Composition API** : `ref`, `computed`, `watch`, `onMounted`
- **Typage** : TypeScript strict, interfaces explicites
- **Naming** : camelCase fonctions/vars, PascalCase composants
- **Formatage** : Template 2-space indent, script 2-space indent
- **Stores** : `defineStore('app', { state, getters, actions })`

- **API** : Toujours passer par `useApi()` , jamais `fetch()` direct
-

Fin du rapport exhaustif.

Rapport Exhaustif - Aspects Transverses du Projet AtelierMBZ

1. Configuration Backend

1.1 Services Personnalisés (services.yaml)

Service	Rôle	Configuration
App\EventListener\TenantSetterListener	Injecte atelierId sur nouvelles entités	prePersist event
App\Service\ConfigEncryptionService	Chiffre secrets en base via kernel.secret	Injected via binding

1.2 Authentification & Sécurité (security.yaml)

Firewalls :

Firewall	Pattern	Authentificateur	Comportement				
dev	^(/_(profiler	wdt\	css\	images\	js)/`	aucun	Désactivé
public	`/api/(auth/login\	auth/refresh\	auth/google\	health\	public/\	companion/\	photos/file/\
api_root	^/api\$	aucun	Racine API publique				
api	^/api	CookieJwtAuthenticator	Stateless, JWT via cookie ou header				

Access Control :

Chemin	Rôles	Méthode	Notes
^/api/auth/login	PUBLIC_ACCESS	POST	Création session JWT
^/api/auth/refresh	PUBLIC_ACCESS	POST	Renouvellement token

Chemin	Rôles	Méthode	Notes	
<code>^/api/auth/google</code>	PUBLIC_ACCESS	POST	OAuth2 Google (optionnel)	
<code>^/api/health</code>	PUBLIC_ACCESS	GET	Healthcheck public	
<code>^/api/public</code>	PUBLIC_ACCESS	GET/POST	Endpoints client public (booking, suivi, demande)	
<code>^/api/companion</code>	PUBLIC_ACCESS	GET/POST/PATCH	Signature Companion client (PDA réception)	
<code>^/api/photos/file</code>	PUBLIC_ACCESS	GET	Téléchargement photos public	
<code>^/api/webhooks/notifications</code>	PUBLIC_ACCESS	POST	Webhooks Twilio/Mailgun/OVH	
<code>^/api/security/csp-report</code>	PUBLIC_ACCESS	POST	Collecte violations CSP	
<code>^/api/motos/(autocomplete\`</code>	marques)`	PUBLIC_ACCESS	GET	Recherche moto, marques (sans auth)
<code>^/api</code>	IS_AUTHENTICATED_FULLY	tous	Reste API protégé	

Hiérarchie des Rôles :

```

ROLE_SUPER_ADMIN: [ROLE_ADMIN]
ROLE_ADMIN: [ROLE_USER, ROLE_VO_MANAGER]
ROLE_VO_MANAGER: [ROLE_USER]
ROLE_RECEPTIONNAIRE: [ROLE_USER]
ROLE_MECANICIEN: [ROLE_USER]
ROLE_COMPTABLE: [ROLE_USER]
```

1.3 Doctrine ORM (doctrine.yaml)

Paramètre	Valeur	But
<code>dbal.url</code>	<code>%env(resolve:DATABASE_URL)%</code>	PostgreSQL 15
<code>orm.naming_strategy</code>	<code>underscore_number_aware</code>	Colonnes snake_case
<code>orm.auto_generate_proxy_classes</code>	true (dev), false (prod)	Cache Doctrine
<code>orm.filters.tenant_filter.class</code>	<code>App\Doctrine\TenantFilter</code>	Multi-tenant row-level
<code>orm.filters.tenant_filter.enabled</code>	false	Activé dynamiquement via listener

Paramètre	Valeur	But
mapping.App	attribute	Entités avec #[Entity]

TenantFilter : Applique `WHERE atelier_id = :id` sur toutes les requêtes d'entités ayant le champ `atelierId`. Activé via [TenantFilterListener](#) dès `kernel.request`, sauf si utilisateur `ROLE_SUPER_ADMIN`.

1.4 JWT & Authentification (`lexik_jwt_authentication.yaml`)

Paramètre	Valeur	But
secret_key	<code>%env(JWT_SECRET_KEY)%</code>	Clé signage HS256
token_ttl	900 secondes	15 minutes d'expiration
user_id_claim	username	Claim d'identité
encoder.signature_algorithm	HS256	Signature HMAC-SHA256
token_extractors.cookie.enabled	true	Extraction depuis cookie <code>access_token</code>
token_extractors.authorization_header.enabled	true	Extraction header <code>Authorization: Bearer</code>

Payload JWT : `user_id`, `atelier_id`, `role`, `jti`. Révocation stockée en table `revoked_token`.

1.5 API Platform (`api_platform.yaml`)

Paramètre	Valeur
title	Paddock API
version	2.0.0
formats	JSON-LD (<code>application/ld+json</code>), JSON (<code>application/json</code>)
pagination.enabled	true
pagination.itemsPerPage	30
pagination.maxItemsPerPage	200
cache_headers.vary	<code>Content-Type</code> , <code>Authorization</code> , <code>Origin</code>

1.6 Messenger (`messenger.yaml`)

Paramètre	Valeur	But
failure_transport	failed	Queue pour messages échoués
transports.async	<code>%env(MESSENGER_TRANSPORT_DSN)%</code>	Transport principal (default: Doctrine)
transports.failed	<code>doctrine:///default?queue_name=failed</code>	Persistence des failed messages

Paramètre	Valeur	But
transports.sync	sync://	Exécution synchrone (test, dev)

Message Routing :

Message	Destination
SendRappelMessage	async
GeneratePdfMessage	async
ProcessScheduledRappels	async
SendGardiennageRappelMessage	async

1.7 Rate Limiter (rate_limiter.yaml)

Limiter	Limite	Intervalle	Verrouillage	But
public_booking	5 req	1 minute	lock.factory	Prise de RDV publique
public_suivi	60 req	1 minute	lock	Suivi client public
public_companion	60 req	1 minute	lock	Companion client (signature)
public_vo_companion	60 req	1 minute	lock	Companion VO (signature)
public_demande	30 req	1 minute	lock	Demande travaux (validation)
companion_upload	20 req	1 minute	lock	Upload photos intervention

1.8 CORS (nelmio_cors.yaml)

Paramètre	Valeur
origin	%env(CORS_ALLOW_ORIGIN)% (regex)
methods	GET, OPTIONS, POST, PUT, PATCH, DELETE
headers	Content-Type, Authorization, Accept, Origin, X-Requested-With
expose_headers	Link
credentials	true
max_age	3600 secondes

1.9 Mercure (mercure.yaml)

Paramètre	Valeur	But
hubs.default.url	%env(MERCURE_URL)%	Serveur interne (Docker: http://mercure:3000/.well-known/mercure)

Paramètre	Valeur	But
hubs.default.public_url	%env(MERCURE_PUBLIC_URL)%	URL client (/.well-known/mercure en prod)
hubs.default.jwt.secret	%env(MERCURE_JWT_SECRET)%	Clé JWT pour publish/subscribe
hubs.default.jwt.publish	*	Qui peut publier (ici: tous)

1.10 Mailer (mailer.yaml)

```
dsn: %env(MAILER_DSN)%
```

Défaut dev: smtp://mailhog:1025 . En prod: mailgun://KEY:DOMAIN@default ou smtp+tls .

1.11 Framework Core (framework.yaml)

Paramètre	Valeur
secret	%env(APP_SECRET)%
session	true (état HTTP persistant)
serializer.name_converter	camel_case_to_snake_case
test (quand @test)	true

1.12 Workflow (workflow.yaml)

State Machine rendez_vous : Entité RendezVous , propriété statut .

Places (21 au total):

```
en_attente, reserve, confirme, reception, en_cours, en_pause, termine, restitue,
facture, paye, annule, en_attente_pieces, en_attente_reprise, en_gardiennage,
restitue_partiel, no_show
```

Transitions principales (28 total):

- reserver : en_attente → reserve
- confirmer : [en_attente, reserve] → confirme
- reception : confirme → reception
- start_travail : reception → en_cours
- mettre_en_pause / reprendre : boucle en_cours ↔ en_pause
- mettre_en_attente_pieces / reprendre_apres_pieces : gestion manque pièces
- mettre_en_attente_reprise / reprendre_demain : reprise lendemain
- terminer : [en_cours, en_pause] → termine
- restituer : termine → restitue
- restituer_partiel : [en_cours, termine] → restitue_partiel

- `facturer` : [termine, restitue, restitue_partiel] → facture
- `payer` : facture → paye
- `annuler` : [en_attente, reserve, confirme, reception, en_attente_pieces, en_gardiennage] → annule
- `declarer_no_show` / `no_show` : [confirme, reception] → no_show
- `reporter` : [reception, no_show] → confirme
- `passer_gardiennage` / `mettre_en_gardiennage` / `sortir_gardiennage` : gestion gardiennage
- Audit trail activé

2. Configuration Frontend (Nuxt 3)

2.1 Core Settings (`nuxt.config.ts`)

Paramètre	Valeur	But
<code>compatibilityDate</code>	<code>2026-04-14</code>	Version Nuxt compatibilité
<code>ssr</code>	<code>false</code>	Mode SPA (pas de SSR)
<code>modules</code>	<code>@nuxt/ui, @pinia/nuxt</code>	UI composants, state management
<code>css</code>	<code>~/assets/css/main.css</code>	Styles globales
<code>alias['#app-manifest']</code>	<code>./app-manifest.stub</code>	Workaround Vite HMR

2.2 Runtime Config

```
runtimeConfig: {
  public: {
    apiBase: '/api',           // Proxy backend via Caddy :80/:443
    mercureUrl: '/.well-known/mercure', // Mercure client endpoint
  }
}
```

2.3 App Head & Meta

Meta Tag	Valeur
<code>title</code>	Paddock
<code>application-name</code>	Paddock
<code>og:title, og:image</code>	Social preview
<code>twitter:card, twitter:image</code>	Card réseau social
Fonts	Google Fonts Inter (wght 400-800)

Meta Tag	Valeur
Favicon	/branding/paddock-logo-favicon.svg
Apple icon	/branding/paddock-logo-favicon.svg

2.4 CSP & Sécurité (Report-Only)

Content-Security-Policy-Report-Only :

Directive	Valeur	But
default-src	'self'	Tout depuis même origin par défaut
script-src	'self' 'unsafe-inline' 'unsafe-eval'	Vue runtime + Nuxt UI
style-src	'self' 'unsafe-inline' https://fonts.googleapis.com	Styles inline + Google Fonts
font-src	'self' data: https://fonts.gstatic.com	Fonts locales + Google
img-src	'self' data: blob:	Images, SVG data URIs, blob URLs
connect-src	'self'	Fetch/XHR/WebSocket vers /api et Mercure
frame-ancestors	'none'	Pas d'iframage
form-action	'self'	Forms vers même origin
base-uri	'self'	Base URL
object-src	'none'	Pas de plugins
report-uri	/api/security/csp-report	Logs violations au serveur

Other Security Headers :

Header	Valeur
X-Frame-Options	DENY
X-Content-Type-Options	nosniff
Referrer-Policy	strict-origin-when-cross-origin

2.5 Nitro Config (Routes & Proxying)

Route	Cible	But
/api/**	http://localhost:8000/api/**	Proxy API backend
/**	headers CSP Report-Only	Toutes pages HTML

2.6 Dev Settings

Paramètre	Valeur	But
colorMode.preference	dark	Thème sombre par défaut
devtools.enabled	true	Devtools Nuxt
vite.server.watch.usePolling	true	Docker HMR (pas inotify)
vite.server.watch.interval	500ms	Fréquence poll

3. Variables d'Environnement

3.1 Complètes avec Sources & Utilisation

Variable	Défaut	Requis ?	Sources
APP_ENV	dev	Non	docker-compose .env
APP_SECRET	change_me_generate_with_openssl_rand_hex_32	OUI	docker-compose .env
APP_DEBUG	inféré de APP_ENV	Non	Kernel, bootstrap tests
DATABASE_URL	postgresql://atelier:atelier@db:5432/atelier_moto	OUI	docker-compose .env
POSTGRES_DB	atelier_moto	Non	docker-compose
POSTGRES_USER	atelier	Non	docker-compose
POSTGRES_PASSWORD	atelier	Non	docker-compose
DB_PORT	5432	Non	docker-compose
JWT_SECRET_KEY	change_me_generate_with_openssl_rand_hex_64	OUI	docker-compose .env
JWT_PUBLIC_KEY	(vide)	Non	docker-compose
JWT_PASSPHRASE	(vide)	Non	docker-compose
CORS_ALLOW_ORIGIN	`^https?:/(localhost\`	127\0\0\1)(:[0-9]+)?\$`	Non
MAILER_DSN	smtp://mailhog:1025	Non	docker-compose .env
MAILER_HOST	mailhog	Non	.env

Variable	Défaut	Requis ?	Sources
MAILER_PORT	1025	Non	.env
MESSENGER_TRANSPORT_DSN	doctrine://default?auto_setup=0	Non	docker-compose .env
MERCURE_URL	http://mercure:3000/.well-known/mercure	Non	docker-compose .env
MERCURE_PUBLIC_URL	/.well-known/mercure	Non	docker-compose .env
MERCURE_JWT_SECRET	!ChangeThisMercureHubJWTSecretKey!	OUI	docker-compose .env
ADMIN_USERNAME	admin	Non	docker-compose
ADMIN_PASSWORD	Admin123!	OUI (prod)	docker-compose .env
ADMIN_EMAIL	admin@atelier.local	Non	.env
BACKEND_PORT	8000	Non	docker-compose
FRONTEND_PORT	3000	Non	docker-compose
MAILHOG_SMTP_PORT	1025	Non	docker-compose
MAILHOG_UI_PORT	8025	Non	docker-compose
PUBLIC_EDGE_PORT	81	Non	docker-compose
LOCK_DSN	flock	Non	.env
DEFAULT_URI	http://localhost	Non	.env
APP_DOMAIN	localhost	Non	docker-compose
PLAYWRIGHT_BASE_URL	http://localhost ou https://localhost	Non	playwright.config.js env circulaire
TWILIO_DSN	(non défini)	Conditionnel	notifier.yaml, webhook
OVHCLLOUD_DSN	(non défini)	Conditionnel	notifier.yaml, webhook
WEBHOOK_HOST	0.0.0.0	Non	webhook listener Python
WEBHOOK_PORT	9010	Non	webhook listener Python
WEBHOOK_PATH	/github-webhook	Non	webhook listener Python
WEBHOOK_SECRET	(vide)	Non	webhook listener Python

Variable	Défaut	Requis ?	Sources
BRANCH	main	Non	webhook listene Python
PROJECT_DIR	/opt/ateliermbz	Non	webhook listene Python
DEPLOY_SCRIPT	\$PROJECT_DIR/scripts/deploy-server.sh	Non	webhook listene Python

4. Sécurité

4.1 JWT (Lexik Bundle)

Aspect	Détail
Algorithme	HS256 (HMAC-SHA256)
TTL	900 secondes (15 minutes)
Extraction	Cookie access_token OU header Authorization: Bearer ...
Payload	user_id, atelier_id, role, jti (JWT ID unique)
Rafraîchissement	Endpoint /api/auth/refresh (PUBLIC_ACCESS) renvoie nouveau token
Révocation	Via table revoked_token (listé lors parsing token)
Authenticateur custom	App\Security\CookieJwtAuthenticator

4.2 Multi-Tenant Row-Level

TenantFilter (backend/src/Doctrine/TenantFilter.php) :

- Ajoute automatiquement WHERE atelier_id = :id sur toutes requêtes ORM
- Déclenché par TenantFilterListener au kernel.request (priorité -10)
- Résout atelierId via CurrentAtelierResolver (JWT ou cookie)
- ROLE_SUPER_ADMIN bypass : aucun filtre appliqué
- Utilisateurs sans atelierId : filtre avec impossible ID (0)

4.3 Security Headers (backend/src/EventListener/SecurityHeadersListener.php)

Header	Valeur	Mode
X-Frame-Options	DENY	Activé
X-Content-Type-Options	nosniff	Activé

Header	Valeur	Mode
X-XSS-Protection	1; mode=block	Activé
Referrer-Policy	strict-origin-when-cross-origin	Activé
Permissions-Policy	geolocation=(), microphone=(), camera=(self)	Activé
Strict-Transport-Security	max-age=31536000; includeSubDomains	HTTPS seulement
Content-Security-Policy-Report-Only	Report-Only (HTML pages)	Observation phase

4.4 CSRF

Désactivé explicitement pour API stateless (Symfony default).

4.5 Rate Limiting

Configuré via RateLimiterFactory (Symfony natif) :

- Politique `sliding_window` (fenêtre glissante)
- Applicateur: `LockFactory` (flock ou redis en prod)
- Appliqué via contrôleurs (e.g., `PublicBookingController`, `CompanionController`)
- **Public endpoints protégés** : booking (5 req/min), suivi (60), companion (60), vo-companion (60), demande (30)
- **Upload photos** : 20 req/min (anti-spam)

4.6 Webhook Signatures (

[backend/src/Service/WebhookSignatureVerifier.php](#))

Twilio :

- Algorithme: HMAC-SHA1
- Signature header: `X-Twilio-Signature`
- Vérification: `base64(HMAC-SHA1(secret, URL + POST params))`

Mailgun :

- Algorithme: HMAC-SHA256
- Headers: `X-Mailgun-Signature`, `X-Mailgun-Timestamp` (résiliation > 5min)
- Anti-replay: timestamp vérifié

OVH :

- Algorithme: Header `X-Webhook-Token` = secret
- Comparaison simple (pas HMAC)

Tests : [backend/tests/Unit/WebhookSignatureVerifierTest.php](#) (12 tests : Twilio, Mailgun, OVH, invalid, missing)

5. Notifications

5.1 Architecture Multi-Provider

NotificationDispatcher ([backend/src/Service/NotificationDispatcher.php](#)) :

- Dispatcher centralisé
- Essaye providers dans l'ordre : primary d'abord, puis par priority ASC
- Fallback sur provider suivant en cas échec
- Chaque tentative loggée en `NotificationLog`

Channels supportés : `email`, `sms`

Providers :

Provider	Channel	Détail
Symfony Mailer	email	DSN <code>%env(MAILER_DSN)%</code>
Twilio	sms	DSN <code>twilio://SID:TOKEN@default?from=+FROM</code>
OVH SMS	sms	DSN <code>ovhcloud://KEY:SECRET@default?...</code>
Mailgun	email	DSN <code>mailgun://KEY:DOMAIN@default?region=us</code>

5.2 Mercure (Temps Réel)

Aspect	Détail
Technologie	Server-Sent Events (SSE) + WebSocket
Endpoint interne	<code>http://mercure:3000/.well-known/mercure</code>
Endpoint client	<code>/.well-known/mercure</code> (reverse proxy Caddy)
JWT	Signature avec <code>MERCURE_JWT_SECRET</code> , publish: *
Format	JSON via topics (channels Mercure)
Composable client	<code>useNotifications.ts</code> → subscriptions dynamiques

5.3 Templates (`NotificationTemplate` + `NotificationTemplateCatalog`)

Stockage : Base de données `notification_template` par atelier + channel

Rendu : Twig simple (variables remplacées dans `{subject}` et body)

Catalog ([backend/tests/Unit/NotificationTemplateCatalogTest.php](#)) :

- Ensembles de templates par défaut (defaults)
- Bootstrap automatique pour nouvel atelier
- Codes prédéfinis : `rdv_confirme`, `rdv_rappel`, `demande_travaux_supp`, etc.

5.4 Escalade (`NotificationEscalation`)

Délai	Raison
T+5min	1ère escalade (pas de réponse)
T+10min	2ème escalade
T+30min	3ème escalade (critique)

Modifiable par atelier via `ConfigAtelier.delaiEscalade*`.

5.5 Messages Messenger

Message	Transport	Consommateur	But
<code>SendRappelMessage</code>	async	<code>SendRappelHandler</code>	Envoi rappel RDV
<code>GeneratePdfMessage</code>	async	<code>GeneratePdfHandler</code>	Générer PDF ordre de rép., rapport, etc.
<code>ProcessScheduledRappels</code>	async	<code>ProcessScheduledRappelsHandler</code>	Traitement rappels programmés
<code>SendGardiennageRappelMessage</code>	async	<code>SendGardiennageRappelHandler</code>	Relances gardiennage (15j, 30j, 45j, 180j)

6. Tests

6.1 Backend - PHPUnit Unit Tests

Fichier	Cas de test	Couvre
PricingServiceTest	~15	Tarification horaire, MO, pièces, TVA, marges
NotificationDispatcherDTOTest	~10	DTO NotificationMessage, NotificationResult
NotificationTemplateCatalogTest	~8	Bootstrap templates, rendu variables
NotificationTest	~5	Entités Notification, NotificationLog
WebhookSignatureVerifierTest	12	Signature Twilio (HMAC-SHA1), Mailgun (HMAC-SHA256 + anti-replay), OVH
OrdreReparationPolicyTest	~10	Décisions OR : éditable, modifiable, rectificatif

Fichier	Cas de test	Couvre
UserRolesTest	~8	Hiérarchie rôles, permissions métier
RapportInterventionServiceTest	~6	Génération rapports intervention
RendezVousWorkflowServiceTest	~12	Transitions workflow RDV
VOCompanionWorkflowServiceTest	~10	Workflow companion VO (signatures)
VORemiseEnEtatServiceTest	~8	Coûts FRE, marge calcul
AdminValidationServicesTest	~6	Validations admin (tarifs, closures)
MotoCatalogImporterTest	~4	Import catalogue moto CSV
ClauseLegaleCodesTest	~6	Codes clauses légales (RGPD, CGV)
ClauseLegaleVisibilityServiceTest	~8	Filtrages clauses par profil
CurrentAtelierResolverTest	~6	Résolution atelier depuis JWT/cookie
ConfigEncryptionTest	~4	Chiffrement/déchiffrement config
UserMecanicienSyncServiceTest	~6	Sync User ↔ Mecanicien
UserAdminLifecycleTest	~8	Création/suppression user admin
ProcessNotificationEscalationsCommandTest	~6	Commande escalade notifications
CheckDaSivExpiryCommandTest	~4	Vérification DA SIV expiré
RelanceGardiennageCommandTest	~6	Commande relances gardiennage
EssaiRoutierCompletenessTest	~8	Essai routier obligatoire avant terminer
NotificationProviderConfigSanitizerTest	~5	Sanitization config providers
SlotServiceClosureTest	~4	Slots non disponibles (fermetures)
ModeTarificationTest	~6	Modes tarification FORFAIT/HORAIRE/SUR_DEVIS
AdminTemplatePreviewControllerTest	~4	Préview templates PDF admin

Total Unit : ~200 tests

6.2 Backend - PHPUnit Functional Tests

Fichier	Cas de test	Couvre
ApiEndpointsTest	~10	Non-régression endpoints (login, booking public, tracking)
AuthBookingAteliersTest	~6	Booking multi-atelier, isolation tenant
CompanionControllerTest	~12	Signature Companion, photos, transitions
MecanicienControllerTest	~10	Espace mécanicien, RDV mécanicien
FacturationControllerTest	~8	Création factures, export PDF
VOControllerTest	~15	VO achats, ventes, LP PDF, DA SIV

Fichier	Cas de test	Couvre
VORemiseEnEtatControllerTest	~10	FRE workflow, coûts, marges
NotificationProviderApiTest	~12	Webhooks notification (Twilio, Mailgun, OVH)
RdvPrestationCatalogControllerTest	~6	Catalogue prestations par atelier
ClientStatsControllerTest	~4	Stats clients (CA, intervention count)
PontStatusControllerTest	~4	Status ponts en temps réel
VehiculeLookupControllerTest	~6	Recherche véhicule, OCR plaque
AtelierCatalogBootstrapServiceTest	~8	Bootstrap nouvel atelier (templates, config defaults)
NotificationContextTest	~6	Contexte notifications (providers, escalade)
RendezVousWorkflowControllerTest	~10	Transitions workflow API (reception, start_travail, etc.)

Total Functional : ~130 tests

Total Backend Tests : ~330 tests

6.3 Frontend - Vitest

Fichier	Cas de test	Couvre
useApi.test.ts	~8	Composable API (GET, POST, PATCH, erreurs)
useCarteGriseOcr.test.ts	~6	OCR carte grise (parsing VIN, plaque)
voCompanionDraftSync.test.ts	~8	Sync brouillon VO Companion
voVehicleForm.test.ts	~10	Validation formulaire véhicule VO
voRefurbishmentCard.test.ts	~7	Card remise en état (coûts, statuts)
voStore.test.ts	~12	Store Pinia VO (rachats, dépôts, documents)

Total Frontend Unit : ~51 tests

6.4 Frontend - Playwright E2E

Fichier	Cas de test	Couvre
auth.spec.mjs	~4	Login, page publique, redirect unauthenticated
business-flows.spec.mjs	~8	RDV wizard, public booking, slot selection
navigation.spec.mjs	~6	Navigation menu, breadcrumbs
roles-workflow.spec.mjs	~10	Scénarios par rôle (mécanicien, réceptionniste)
vo-pricing-diff.spec.mjs	~4	Calcul tarifs VO (marge vs normal)
vo-companion-flow.spec.mjs	~8	Companion VO (signatures, documents)
notifications.spec.mjs	~6	Mercure live updates, notifications

Fichier	Cas de test	Couvre
notification-providers.spec.mjs	~5	Providers email/SMS actifs
non-regression.spec.mjs	~12	Non-régression : pages critiques
helpers.mjs	1	Utilitaires (loginAsAdmin, etc.)

Total Frontend E2E : ~63 tests

Total Tests Frontend : ~114 tests

7. Docker Compose

7.1 Services (docker-compose.yml)

Service	Image	Port	Volumes	Roles
db	postgres:15-alpine	5432 (internal)	postgres_data:/var/lib/postgresql/data	DB principal
php	Custom (Dockerfile.backend)	8000	app code, logos, photos, signatures	API backend
worker	Custom (Dockerfile.backend)	-	app code, logos, photos, signatures	Consumer Messenger
nuxt	Custom (Dockerfile.frontend)	3000	-	Frontend SPA
caddy	caddy:2.9-alpine	80, 443, 81	Caddyfile (ro), caddy_data, caddy_config	Reverse proxy
mercure	dunglas/mercure:v0.18	3000 (internal)	mercure_data, mercure_config	Notifications temps réel
mailhog	axllent/mailpit:v1.24	1025 (SMTP), 8025 (UI)	-	Capture emails dev

7.2 Healthchecks

Service	Check
db	pg_isready -U atelier -d atelier_moto (5s interval, 3s timeout, 10 retries)
php	dépend_on: db healthcheck
worker	dépend_on: db healthcheck
nuxt	dépend_on: php

Service	Check
caddy	dépend_on: php, nuxt, mercure

7.3 Environment Variables Transmis

Service	Clés
db	POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD
php	DATABASE_URL, APP_ENV, APP_SECRET, JWT_SECRET_KEY, JWT_PUBLIC_KEY, JWT_PASSPHRASE, CORS_ALLOW_ORIGIN, MAILER_DSN, MESSENGER_TRANSPORT_DSN, MERCURE_URL, MERCURE_PUBLIC_URL, MERCURE_JWT_SECRET, ADMIN_USERNAME, ADMIN_PASSWORD
worker	DATABASE_URL, APP_ENV, APP_SECRET, MAILER_DSN, MESSENGER_TRANSPORT_DSN
nuxt	NUXT_PUBLIC_API_BASE
caddy	APP_DOMAIN
mercure	MERCURE_PUBLISHER_JWT_KEY, MERCURE_SUBSCRIBER_JWT_KEY, MERCURE_EXTRA_DIRECTIVES (CORS, anonymous, subscriptions)

7.4 Volumes Persistants

Volume	Contenu	Utilisé par
postgres_data	Base données PostgreSQL	db
backend_logos	Logos ateliers	php, worker
backend_photos	Photos interventions/véhicules	php, worker
backend_signatures	Signatures numériques	php, worker
caddy_data	Certificats TLS, état	caddy
caddy_config	Config dynamique	caddy
mercure_data	Événements persistants	mercure
mercure_config	Config Mercure	mercure

7.5 Networking

- Réseau `bridge` par défaut
- Services accessibles par hostname (e.g., `db:5432`, `php:8000`)
- Port 81 : edge public (public-facing)

7.6 Docker-Compose Préprod (docker-compose.preprod.yml)

Différences :

Aspect	Dev	Préprod
Env obligatoires	Defaults fournis	<code>\${VARIABLE:?must be set}</code>
Volumes	Bind-mounted code	Volumes named persistants
Mailhog	<code>profiles: ["dev-only"]</code>	Désactivé
Secrets	.env fichier	Env variables d'orchestration

8. Caddy Reverse Proxy (Caddyfile)

8.1 Ports & Listeners

Port	Nom	But
:80 , :443	Interne	Back-office complet (admin, planning, mécanicien, VO)
:81	Public edge	Client-facing uniquement (booking, suivi, demande, mentions)

8.2 Main Listener (:80 , :443)

Route	Cible	Compression	Détail
<code>/.well-known/mercure</code>	<code>mercure:3000</code>	zstd, gzip	Mercure hub proxy
<code>/api/*</code>	<code>php:8000</code>	zstd, gzip	Backend API
<code>/uploads/*</code>	<code>php:8000</code>	zstd, gzip	Téléchargements (photos, logos)
<code>/</code>	<code>nuxt:3000</code>	zstd, gzip	Frontend SPA

8.3 Public Edge Listener (:81)

Intention : Exposer UNIQUEMENT les pages client publiques (booking, suivi, demande, mentions, politique).

Route	Cible	Compression	Matcher
<code>/api/*</code>	<code>php:8000</code>	zstd, gzip	Public API endpoints (booking, suivi, demande, photos)
<code>/uploads/*</code>	<code>php:8000</code>	zstd, gzip	Téléchargements publics
<code>/.well-known/mercure</code>	<code>mercure:3000</code>	zstd, gzip	Mercure (temps réel suivi)
<code>/public/booking*</code>	<code>nuxt:3000</code>	zstd, gzip	Prise RDV publique
<code>/public/suivi</code> , <code>/public/suivi/*</code>	<code>nuxt:3000</code>	zstd, gzip	Suivi RDV client

Route	Cible	Compression	Matcher
/public/demande , /public/demande/*	nuxt:3000	zstd, gzip	Validation demande travaux
/public/mentions-legales*	nuxt:3000	zstd, gzip	Mentions légales
/public/politique-confidentialite*	nuxt:3000	zstd, gzip	Politique confidentialité
/_nuxt/, /__nuxt/, /favicon.ico , /branding/*, /robots.txt	nuxt:3000	zstd, gzip	Assets Nuxt
/	redir 302 → /public/booking	-	Root redirect
Catch-all	404 response	-	Tout le reste rejeté

9. CI/CD (.github/workflows/)

Status : Aucun workflow détecté — pas de répertoire .github/workflows/ .

Implication : Déploiement manuel ou via webhook listener Python (scripts/github-webhook-listener.py).

Webhook listener config :

Variable	Défaut
WEBHOOK_HOST	0.0.0.0
WEBHOOK_PORT	9010
WEBHOOK_PATH	/github-webhook
WEBHOOK_SECRET	(vide)
BRANCH	main
PROJECT_DIR	/opt/ateliermbz
DEPLOY_SCRIPT	\$PROJECT_DIR/scripts/deploy-server.sh

10. Résumé des Aspects Transverses

10.1 Authentification & Autorisation



```
TenantFilter (WHERE atelier_id = :id)
      ↓
RolePermissionVoter (PERM_*)
      ↓
Access-control rules (patterns + roles)
```

10.2 Multi-Tenant Isolation

- **Row-level** : TenantFilter Doctrine
- **SUPER_ADMIN bypass** : explicite et audité
- **Default value** : CurrentAtelierResolver (JWT ou cookie)
- **Fallback** : impossible ID (0) si non résolvable

10.3 Notifications

```
NotificationDispatcher
      ↓
Provider Selection (priority order, fallback)
      ↓
Channel Routing (email, sms)
      ↓
Twilio / OVH / Mailgun / Symfony Mailer
      ↓
NotificationLog (audit)
```

Temps réel : Mercure (SSE/WebSocket) sur `/api/notifications` + `useNotifications.ts`

10.4 Data Security

- **Secrets** : `APP_SECRET`, `JWT_SECRET_KEY`, `MERCURE_JWT_SECRET` (requis)
- **Chiffrement** : `ConfigEncryptionService` (base `kernel.secret`)
- **Snapshots RGPD** : Colonnes `snap_*` sur OR, factures, LP
- **Rate limiting** : 6 limiters (sliding window, cache storage)

10.5 Deployment Topology

```
PROD/PREPROD
(PORT 80, 443)
      ↓
Caddy:2.9
  /      \
php:8000  nuxt:3000
      ↓
```

PostgreSQL:15



Worker
(Messenger
Consumer)

REAL-TIME



Mercure:0.18

10.6 Test Coverage

Type	Framework	Count	Focus
Unit	PHPUnit	~200	Services, DTO, policies
Functional	PHPUnit	~130	Controllers, workflows, webhooks
E2E	Playwright	~63	Business flows, auth, roles
Frontend Unit	Vitest	~51	Composables, stores

Conclusion

AtelierMBZ implémente une **architecture multi-tenant row-level cohérente** avec JWT stateless, rate limiting, webhook signature verification, et notifications multi-canaux (email, SMS, Mercure temps réel). La sécurité repose sur TenantFilter Doctrine, CSP Report-Only, headers de sécurité, et audit trail. Les tests couvrent le flux métier principal (RDV → réception → intervention → restitution) ainsi que le module VO (rachat → FRE → revente). Deployment via Docker Compose avec Caddy comme reverse proxy, PostgreSQL 15, et Mercure pour les mises à jour en temps réel.